



ANM-Shop

BEAUTY FOR EVERYDAY RITUALS

PROJECT DOCUMENTATION

ANM-Shop — online beauty store

Architecture (HLD), low-level design (LLD), user and API workflows, API and data reference, security, operations and roadmap.

EDITION

4 · up to pull request #34

DATE

10 October 2026

STACK

React 19 · Express 5 ·
MongoDB

LIVE

anm-shop.vercel.app

Table of contents

1. Project overview

Who uses it · What it does · Where it runs

2. Technology stack

Frontend · Backend · External services · Folder structure

3. High-level design (HLD)

System architecture · Deployment · Design decisions · Quality attributes

4. User workflows

Shopper journey · Sign-up · Sessions · Password reset · Checkout · Orders & reviews · Support · Admin

5. API workflows

Sign-up · Sign-in · Password reset · Payment · Webhook · Cancel and return · Chat assistant

6. Low-level design (LLD)

Request pipeline · Backend modules · Frontend architecture · Data model · Pricing engine · State machines · Fulfilment · Cancellations · AI internals

7. API reference

All 60 endpoints by area

8. Data model reference

Fields and rules for all 10 collections

9. Security

Headers, auth, codes, bots, payments, input, privacy

10. Rate limits

11 limiters

11. Frontend reference

Routes · SEO · Performance · Consent · Look and feel

12. Configuration and operations

Environment variables · Local setup · Scripts · Start-up · Checklist

13. Project history

Pull requests #1–#34

14. Known limitations and next steps

What to improve next

How to read this document

Parts 1–3 give the big picture. Part 4 follows real people through the app; Part 5 shows the same journeys as API calls; Part 6 opens up the internals. Parts 7–12 are reference material to look things up. Diagram colours: dark boxes are starting points or people, white boxes are steps inside ANM-Shop, gold boxes are outside services or decisions, green boxes are good outcomes, red boxes are failures, and blue boxes are information or alternatives.

Project overview

ANM-Shop is a full-stack online beauty store for customers in India. Shoppers browse skincare, makeup, haircare and body products, pay with Razorpay, track their orders, write reviews and get help through support tickets or an AI chat assistant. Admins manage products, orders, coupons, reviews, tickets and customers from one dashboard.

60

REST API endpoints across 12 route areas

10

MongoDB data models (plus a `migrations` marker collection)

20

frontend routes served by 19 page components

77

automated tests, run by GitHub Actions on every pull request: 54 for the API on an in-memory database, 23 for screens in a simulated browser

11

rate limiters protecting sign-in, email, payments, chat and more

7

default categories, managed by admins; 44 demo products in the seed data

Who uses it

Shoppers

Guests can browse, search and fill a cart. Signing up (with email verification) unlocks checkout, order history, bills, reviews, coupons, support tickets and the chat assistant.

Admins

Accounts with `role: admin` see the admin dashboard: sales statistics in India time, products, categories, orders (with cancellations, returns and refunds), coupons, reviews, tickets, users and roles, and a search across orders, payments and customers. The first admin is made with the `promote-admin` script; after that admins manage roles in the dashboard.

What it does

Shopping

Admin-managed categories, relevance-ranked search, up to 6 photos per product, Bad / Good / Excellent reviews, a per-account cart that refreshes prices and stock.

Accounts

Sign-up with a 6-digit email code, sign-in, change password (signs out other devices), forgot / reset password by email link, self-service account deletion.

Checkout & payment

Server-side pricing in paise, ₹49 shipping under ₹499, one coupon per order, Razorpay Checkout, signature checks, webhook backup, automatic refunds.

Orders

Order history with price breakdown, PDF bills and e-bill emails; customers cancel unshipped orders and admins mark returns, both refunded automatically.

Support

Tickets with threaded replies and email notices; a Gemini-powered assistant that reads the customer's own orders, payments and tickets, finds products and adds them to the cart.

Launch-ready extras

Privacy, terms and returns pages, SEO tags, sitemap, consent-gated analytics, a full Content-Security-Policy, dark mode, lazy pages (the footer no longer jumps while they load), resized images, spam protection, API and frontend tests run on every pull request, and business details and a grievance officer that appear once filled in.

Where it runs

Part	Address	Hosting
Storefront (React app)	https://anm-shop.vercel.app	Vercel (static build + CDN)
API (Express)	https://anm-shop-api.onrender.com	Render web service, free plan, Node 22
Database	MongoDB Atlas	Managed replica set (needed for transactions)
Source code	github.com/MOHILKUMAR/ANM-Shop	<code>main</code> branch, up to pull request #34
Releases	GitHub tags	<code>v0.9.0</code> : pre-launch (Razorpay still in test mode)

Scope of this document. Everything here was taken from the code as of 10 October 2026, up to pull request #34 (CI, India time in emails, and business details on the site and the bills): `backend/` , `frontend/` , `render.yaml` and `frontend/vercel.json` . Secret values from `.env` files are not included; only the variable names are listed.

PART 2

Technology stack

Frontend

Technology	Version	Used for
React + React DOM	19.2.7	User interface, context providers, lazy pages
React Router DOM	7.18.4	Client-side routing (20 routes)
Vite + @vitejs/plugin-react	8.1.4 / 6.0.3	Dev server (proxies <code>/api</code> to port 5000) and production build
Tailwind CSS (@tailwindcss/vite)	4.3.2	Styling, plus custom brand colour variables and dark mode overrides
jsPDF	4.2.1	PDF bill generated in the browser (loaded only when used)
write-excel-file	4.1.1	Admin user list export to <code>.xlsx</code> (loaded only when used)
@vercel/analytics, @vercel/speed-insights	2.0.1 / 2.0.0	Cookie-free visit and speed data, only after consent
ESLint (flat config)	10.7.0	Linting with react-hooks and react-refresh rules

Technology	Version	Used for
Vitest · React Testing Library · jsdom	5.0.3 · 16.3.3 · 29.0.2 (dev)	Component tests of the admin Orders, Tickets and Search tabs against a faked API (<code>npm test</code>)

Production builds first check that the Content-Security-Policy in `vercel.json` matches the inline theme script (`scripts/csp-hash.mjs`).

Backend

Technology	Version	Used for
Node.js	22 (Render), 22.19 locally	Runtime (CommonJS modules)
Express	5.2.1	HTTP server and routing
Mongoose	9.7.3	MongoDB models, validation, transactions
jsonwebtoken	9.0.3	Session tokens (JWT, 30 days)
bcryptjs	3.0.3	Password hashing (cost 12)
helmet / cors	8.3.0 / 2.8.5	Security headers and origin allow-list
express-rate-limit	8.7.0	11 rate limiters (in-memory store)
multer	2.4.0	Image upload in memory (5 MB, JPEG/PNG/WebP)
cloudinary	2.10.0	Product image storage (max 1600 × 1600)
razorpay	2.9.6	Orders, payment fetch / capture, refunds
@google/genai	2.24.0	Gemini chat assistant with function calling
nodemailer	10.0.14	Gmail SMTP fallback for email
dotenv · nodemon	17 · 3 (dev)	Environment loading, auto-restart in development
node:test · supertest · mongodb-memory-server	Node 22 · 7.3 · 11.3 (dev)	API tests against an in-memory MongoDB replica set with a fake Razorpay

External services

Service	Role in ANM-Shop
Vercel	Hosts the built React app; adds security headers; rewrites <code>/sitemap.xml</code> to the API and every other path to <code>index.html</code> ; Web Analytics and Speed Insights.
Render	Runs the Express API from <code>render.yaml</code> (free plan, health check on <code>/</code>).
MongoDB Atlas	Stores all data. Payment fulfilment uses multi-document transactions, which need a replica set.
Razorpay	Payment window in the browser (UPI, cards, net banking, wallets), Orders API, captures, refunds, <code>payment.captured</code> webhook.
Cloudinary	Product photo upload (folder <code>anm-shop/products</code>) and resized delivery (<code>f_auto</code> , <code>q_auto</code> , <code>w_...</code>).
Google Gemini	AI support assistant: model <code>gemini-3.8-flash</code> , fallback <code>gemini-3.5-flash</code> .

Service	Role in ANM-Shop
Resend · Gmail	Transactional email (codes, reset links, e-bills, ticket updates). Resend when configured, Gmail SMTP otherwise.
placeholder.co	Placeholder images for the 44 seeded demo products.

Folder structure

backend/

- `app.js` middleware, routes, error handler; `index.js` start-up
- `config/` MongoDB connection, Cloudinary setup, business details for the e-bill
- `test/` API tests (`npm test`)
- `controller/` 12 controllers (auth, product, category, review, order, payment, coupon, ticket, chat, analytics, search, sitemap)
- `middleware/` protect, admin, rate limiters, spam guard
- `model/` 10 Mongoose schemas
- `routes/` one router per API area
- `utils/` pricing, categories, order refunds, Razorpay client, email, e-bill, AI assistant, tickets, reviews, migrations
- `data/` + `seed.js` demo catalogue; `promoteAdmin.js`

frontend/

- `index.html` default SEO tags, preconnects, theme script
- `public/` icons, og-image.png, robots.txt, web manifest
- `src/App.jsx` router and layout; `src/index.jsx` providers
- `src/pages/` 19 pages, and tests for the Contact and policy pages and the PDF bill
- `src/components/` shared UI, product gallery and 7 admin tabs; `*.test.jsx` component tests
- `src/context/` AuthProvider, CartProvider
- `src/data/` contact info and business details, review maths, ticket labels
- helpers: `api.js` , `useCategories.js` , `consent.js` , `imageUrl.js` , `money.js` , `usePageMeta.js`
- `vercel.json` , `vite.config.js` , `eslint.config.js` , `scripts/csp-hash.mjs`

The project root also holds `README.md` , `package.json` (runs both apps with `concurrently`), `render.yaml` (Render blueprint), `.github/workflows/ci.yml` (the CI checks) and `docs/` (this document and its sources).

High-level design (HLD)

ANM-Shop is a classic three-tier web application: a single-page React app served from a CDN, one stateless REST API, and one MongoDB database, with payments, images, AI and email handled by managed services.

3.1 System architecture

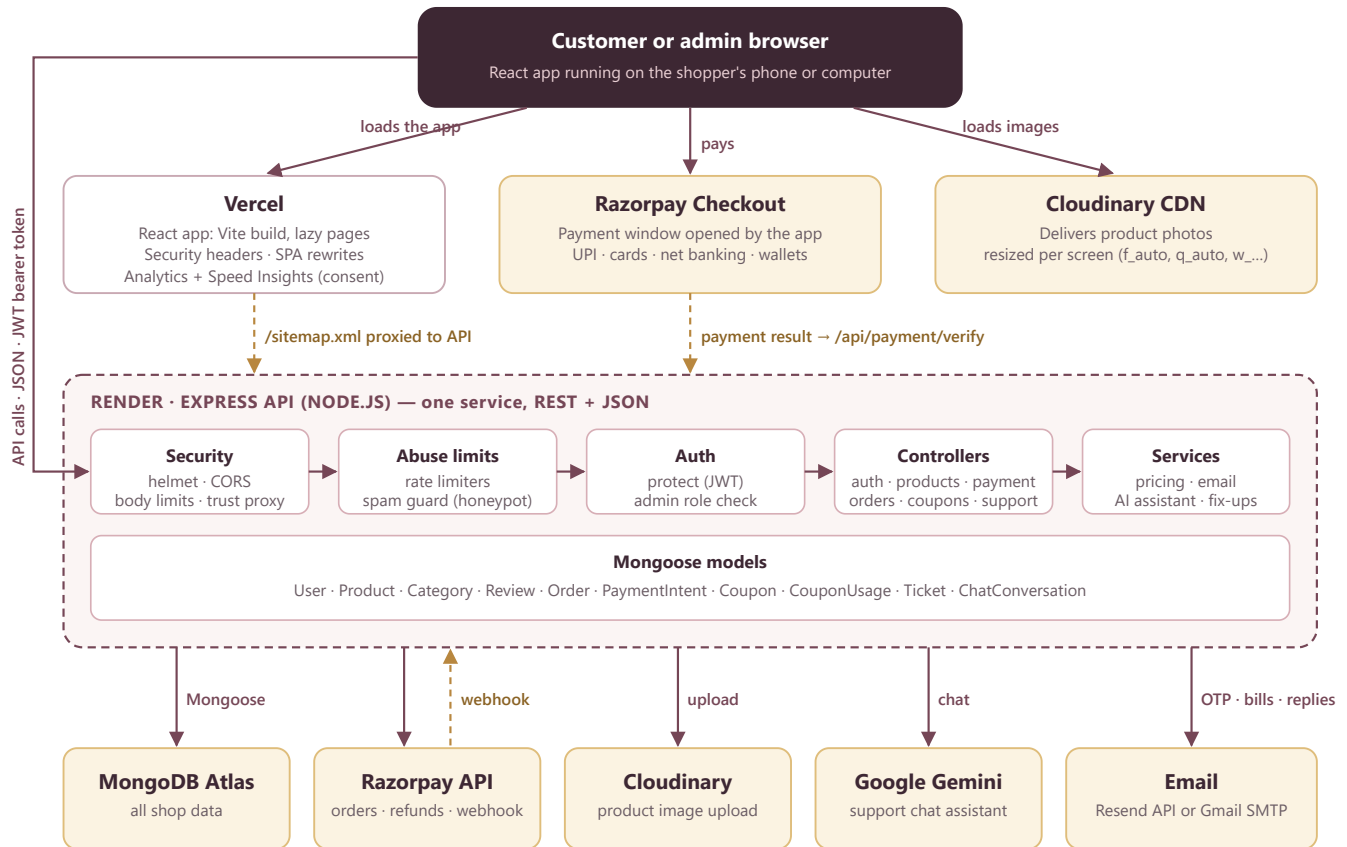


Figure 3.1 — System architecture: who talks to whom

- The browser loads the app from Vercel.** Vercel serves the static Vite build with year-long caching for hashed assets, security headers and an SPA fallback, so every page URL returns `index.html`.
- The app calls the API directly** at `VITE_API_URL` with JSON requests. Signed-in requests carry `Authorization: Bearer <JWT>`. CORS only allows origins listed in `FRONTEND_URL`.
- The API is the source of truth.** It re-prices every cart from the database, checks every payment with Razorpay and never trusts amounts from the browser.
- Payments** open in the Razorpay window. The browser sends the result to `/api/payment/verify`. Razorpay also calls the API's webhook, so an order is still created if the browser disconnects.
- Images** are uploaded by the API to Cloudinary and served to browsers from Cloudinary's CDN in the size each screen needs.
- Email and AI** are outbound calls from the API: Resend or Gmail for email, Google Gemini for the chat assistant.

3.2 Deployment

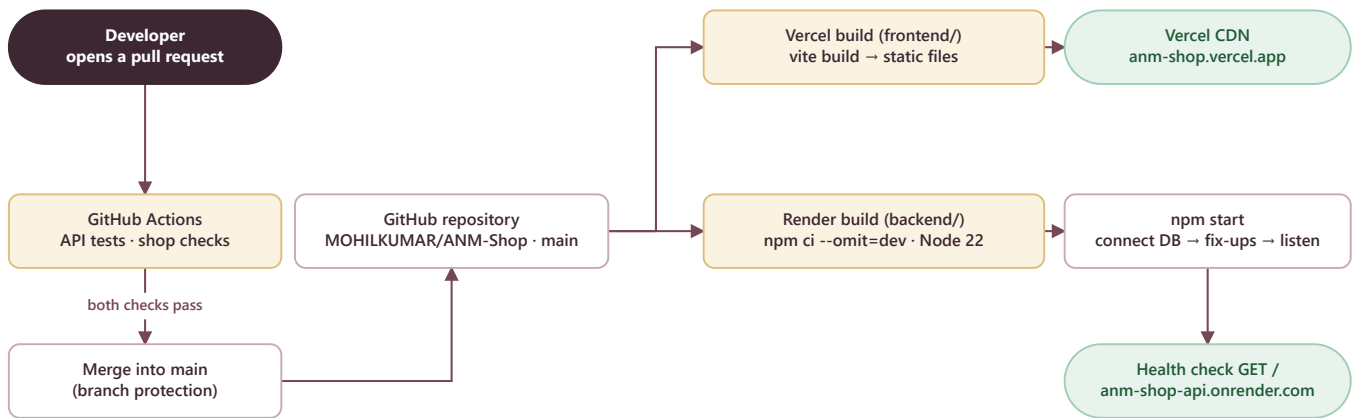


Figure 3.2 — How code reaches production

Every pull request first runs two GitHub Actions checks (`.github/workflows/ci.yml` , Node 22, no secrets): **API tests**, and the shop's **lint, tests and build** (the build also checks the Content-Security-Policy hash). Branch protection on `main` keeps the merge button disabled until both pass; an admin can bypass it only by ticking a box. Each merge runs the checks again on `main` . Releases are tagged on GitHub: `v0.9.0` marks the pre-launch code, and `v1.0.0` is planned for the day live payments start.

Both halves deploy from the same GitHub repository. Vercel builds `frontend/` ; Render follows `render.yaml` : root `backend` , build `npm ci --omit=dev` , start `npm start` , health check `GET /` , `NODE_ENV=production` , `TRUST_PROXY=1` and a generated `JWT_SECRET` . Other secrets are entered once in the Render dashboard. On start the API connects to MongoDB, runs two safe fix-ups and then starts listening. In production it refuses to start if `JWT_SECRET` is weak or `FRONTEND_URL` is missing or not HTTPS.

3.3 Key design decisions

Server-side pricing in paise

Prices, shipping and coupons are computed only on the server, in whole paise, from database values. The same calculation runs for the quote and for the payment, so the amount shown is the amount charged.

Exactly-once orders

Fulfilment runs in a MongoDB transaction that only proceeds while the payment record is `pending` ; `Order.paymentId` is unique. Verify and the webhook can race safely.

Refund instead of overselling

Stock and coupon limits are claimed atomically at fulfilment. If an item sold out or a coupon ran out while the customer was paying, the payment is refunded in full automatically. Cancelled and returned orders are refunded the same way, claimed atomically so they are never refunded twice.

Stateless sessions, revocable

JWTs need no server session store. Changing or resetting a password sets `passwordChangedAt` , which invalidates every older token.

Cart in the browser

Carts live in `localStorage`, one per account on a shared device, and are refreshed from `/api/products/lookup` on the cart and checkout pages.

Privacy by default

Analytics load only after consent and strip query strings except `category` and `page` . Email codes and reset tokens are stored only as hashes.

3.4 Quality attributes

Attribute	How it is achieved	Current limits
Security	helmet, full Content-Security-Policy, CORS allow-list, JWT + bcrypt, hashed codes and tokens, HMAC payment checks, spam guard, 11 rate limiters, input limits, HTML-escaped emails	Token kept in localStorage
Reliability	webhook backup, transactions, automatic refunds with retry (admins emailed and a Refund problems list when one fails), idempotent start-up fix-ups, Gemini retries and fallback model, 54 API and 23 frontend tests run on every pull request	Failed checkout refunds need a manual refund in Razorpay
Performance	CDN, lazy pages, on-demand jsPDF / Excel / Razorpay scripts, Cloudinary resizing, text-indexed search plus a partial-word scan, cached categories, immutable asset caching	Render free plan; the partial-word scan reads every product (fine for hundreds)
Scalability	Stateless API (can add instances), indexed queries, paged admin lists	In-memory rate-limit counts are per instance; the admin user list loads every account
Usability	Dark mode, skeleton loaders, clear error text, WCAG-AA colour tuning, mobile layouts, self-service cancel and account deletion	Returns are requested through a ticket

User workflows

How shoppers and admins move through the app. Each diagram shows what the person sees; Part 5 shows the matching API calls.

4.1 Shopper journey

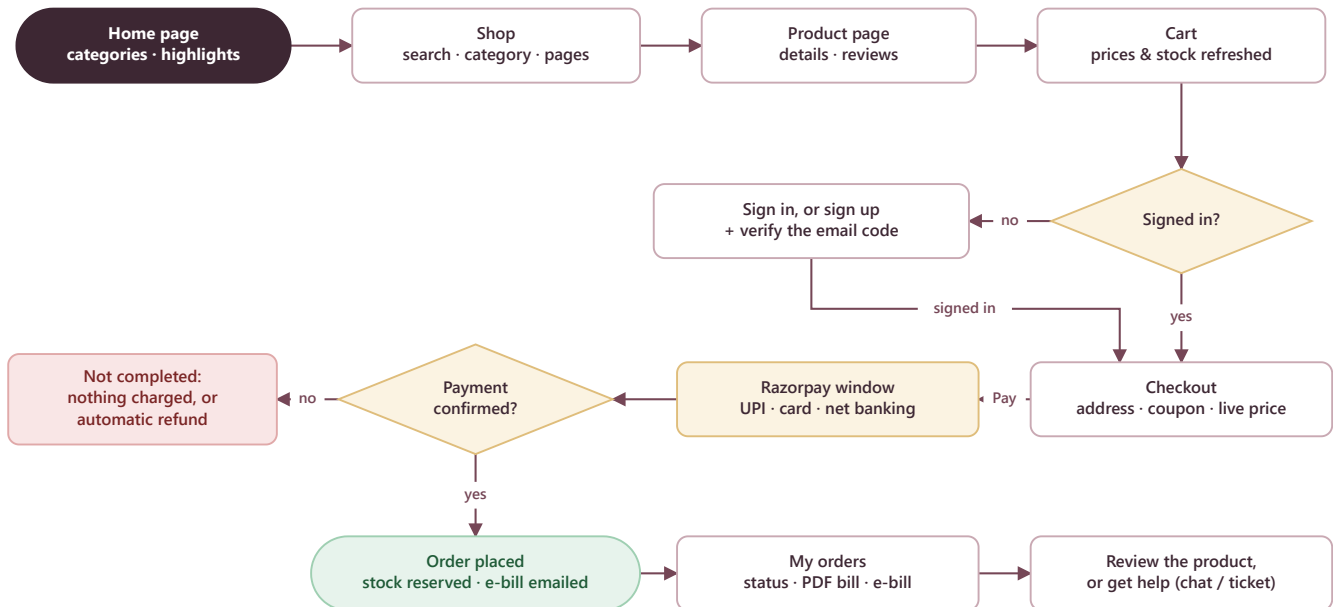


Figure 4.1 — From landing page to a delivered order

- 1 **Browse.** The home page shows a tile for each category (managed by admins). The Shop page has a search box (250 ms debounce; whole-word matches ranked by relevance first, then products containing the word inside a longer one, so "lip" also finds "Lipstick"), a category filter and pages of 24; the filter and page live in the URL. An old link to a renamed or removed category shows the whole shop with a note.
- 2 **Choose.** A product page shows a photo gallery (up to 6), price, stock, an overall rating and reviews. "Add to cart" stores a copy of the product; quantity is capped at stock and 99.
- 3 **Review the cart.** Opening the cart checks today's price and stock and lists changes, e.g. "now costs ₹X (was ₹Y)", "Only N left", "no longer sold".
- 4 **Sign in.** Checkout needs an account. A guest cart is merged into the account cart on sign-in.
- 5 **Pay and track.** After payment the order appears in *My orders* with its status, breakdown, a downloadable PDF bill and the e-bill email. Until it ships, the customer can cancel it there for a full refund.

4.2 Sign-up and email verification

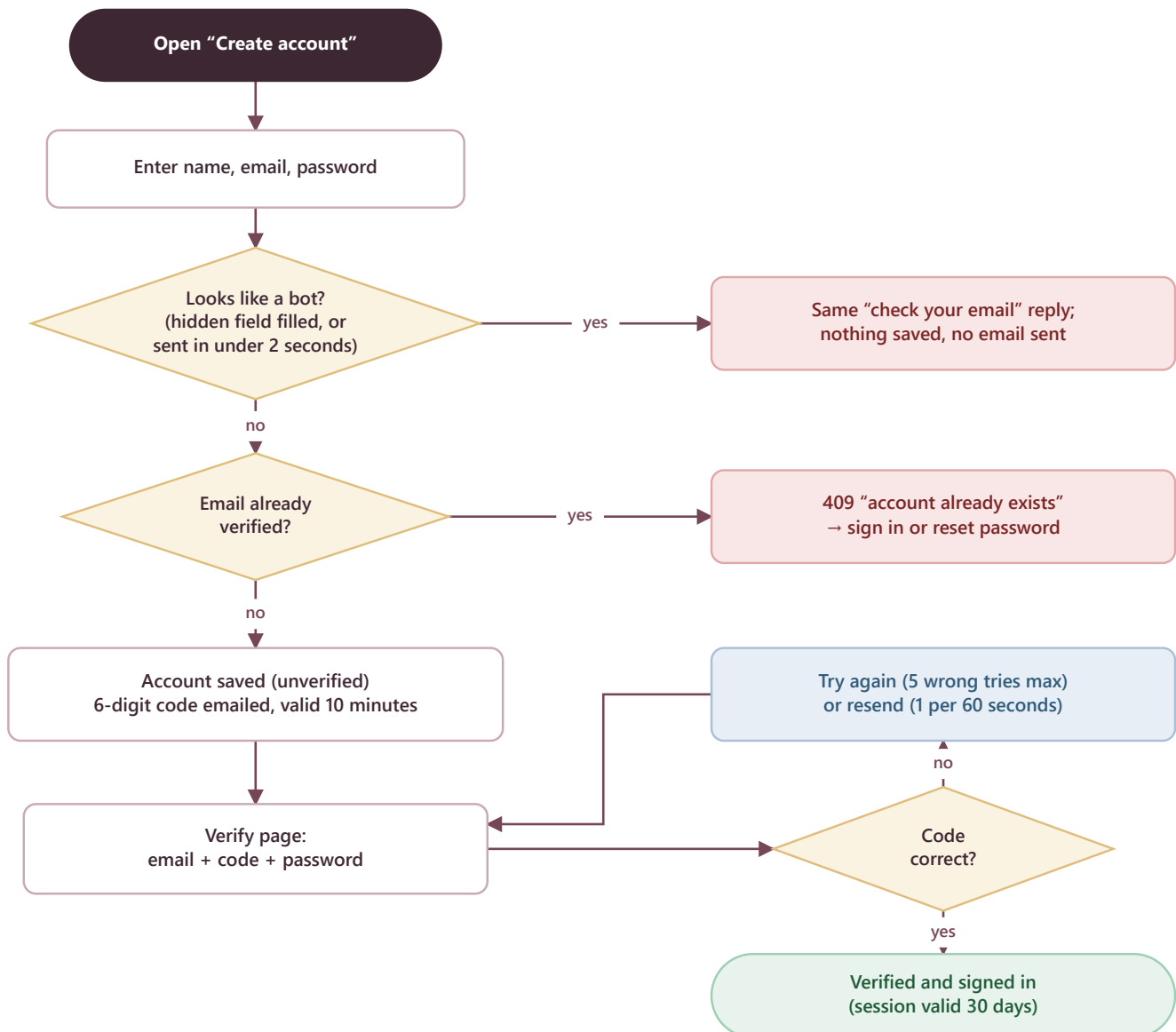


Figure 4.2 — Creating and verifying an account

- The form has a hidden `leaveBlank` field and sends how long it was open. Bots get the same reply as people, but nothing is saved.
- Re-registering an *unverified* email replaces its name and password, so whoever controls the inbox ends up owning the account. Verifying requires the code *and* the password.
- Sign-in is refused (403) until the email is verified; the app then sends the person to the verify page.

4.3 Sign in, sessions and password change

- Sign-in returns a JWT valid for 30 days, kept in `localStorage` (`userInfo`). Expired tokens are dropped when the app loads.
- Any 401 from the API fires a "session expired" event and signs the person out on that device.
- **Change password** (Account page) needs the current password; a wrong one returns 400 so the person is not logged out. Success signs out all other devices, emails a notice and returns a fresh token.
- **Delete account** (Account page) needs the password and a confirmation tick. In one step it removes the account, reviews and chat history, closes open tickets (kept as records, like orders and payments) and switches off coupons meant only for that customer. It is refused while an order is pending or shipped, while a refund to them is still

pending or failed, while a payment is still being confirmed or a failed checkout is being refunded (checkouts touched in the last 24 hours), and for admins (another admin must make them a customer first). Admins deleting a customer get the same checks.

4.4 Forgot and reset password

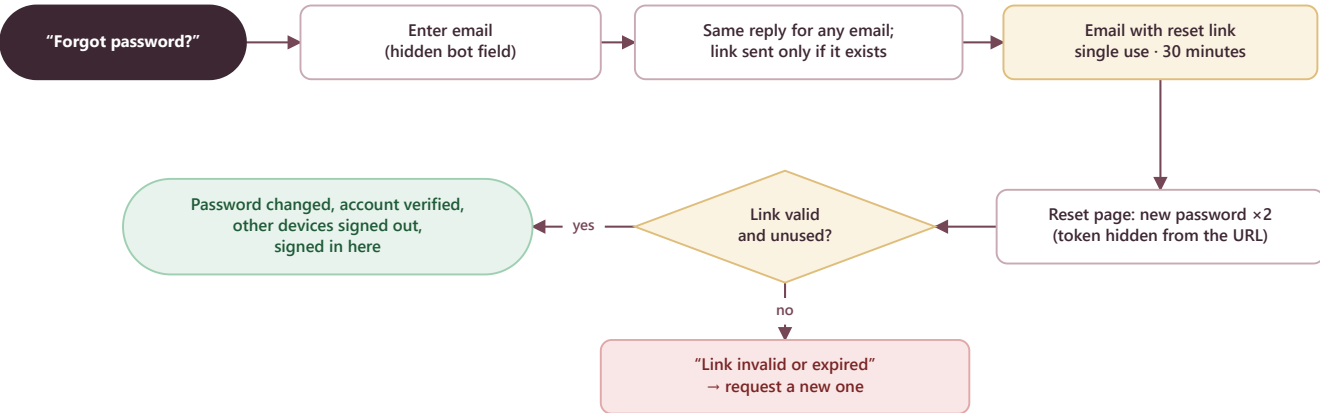


Figure 4.3 — Resetting a forgotten password

The reply is the same whether or not the email has an account, so the form cannot be used to discover customers. Links are single-use, valid for 30 minutes, and at most one is sent per minute. Resetting also verifies the account.

4.5 Cart, checkout and payment

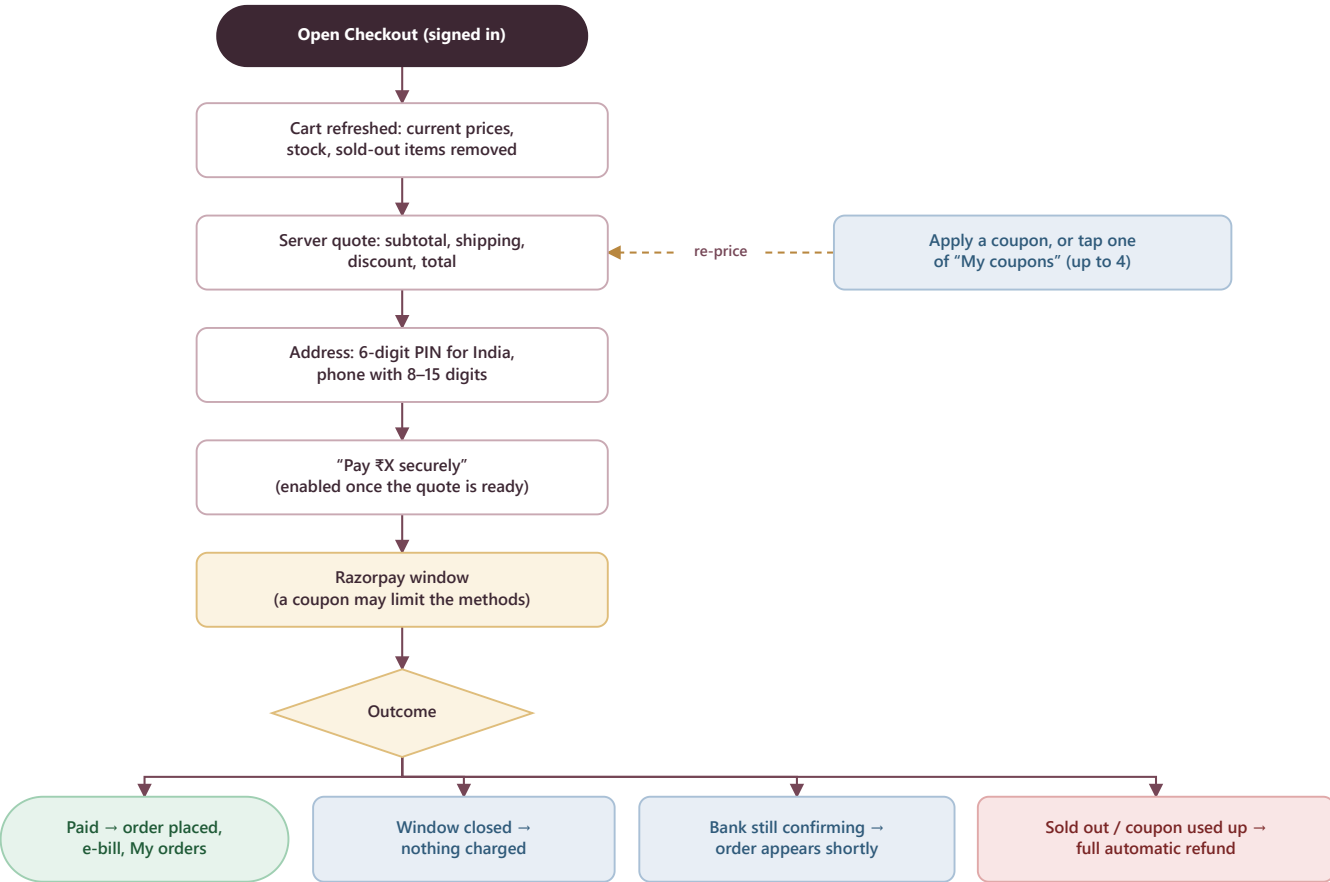


Figure 4.4 — The checkout page and its possible outcomes

Rule	Detail
Shipping	₹49 when the items subtotal is below ₹499; free otherwise (both configurable on the server).
Coupons	One per order. The customer's usable coupons appear as one-tap suggestions (up to 4). An invalid coupon is reported and the price is shown without it.
Address	Full name, street, city, PIN / postal code, country (default India), phone. Indian PINs must be 6 digits not starting with 0; phones need 8–15 digits.
Payment methods	All Razorpay methods, unless the coupon limits them (e.g. UPI only); the window then shows only those.
After paying	Success clears the cart and opens My orders. If confirmation fails because of the network, the app says the order will appear shortly and not to pay again.

4.6 Orders, bills and reviews

- **My orders** lists each order with items, subtotal, shipping (or "Free"), discount with coupon code, total paid, delivery details and e-bill status.
- **Download bill (PDF)** builds an A4 receipt in the browser: header, order and payment IDs, address, item table and totals (`ANM-Shop-Bill-<id>.pdf`); for a cancelled or returned order it adds a "Refunded" (or "Refund in progress") line with the amount. Once the business details are filled in, a "Sold by" block names the seller (legal name, address, GSTIN, phone). The totals, any refund line and the closing line always stay together above the bottom margin.
- **Resend e-bill** emails the receipt again (paid orders that were not cancelled or returned, 5 per hour). The e-bill shows the order time in India time, and the seller once the business details are filled in.
- **Cancel order:** shown while an order is pending. The full amount, shipping included, is refunded to the original payment method; stock goes back and the coupon use is returned. The order then shows its refund ("issued" or "in progress"). Shipped orders are returned through a support ticket; the admin marks the return and the refund follows.
- **Reviews:** one per customer per product, rated Bad, Good or Excellent with a 10–1000 character comment. Submitting again edits it. A "Verified buyer" label appears if the customer ordered the product (and goes once every order of it is cancelled). Public names look like "Priya S.".

4.7 Support: tickets and the chat assistant

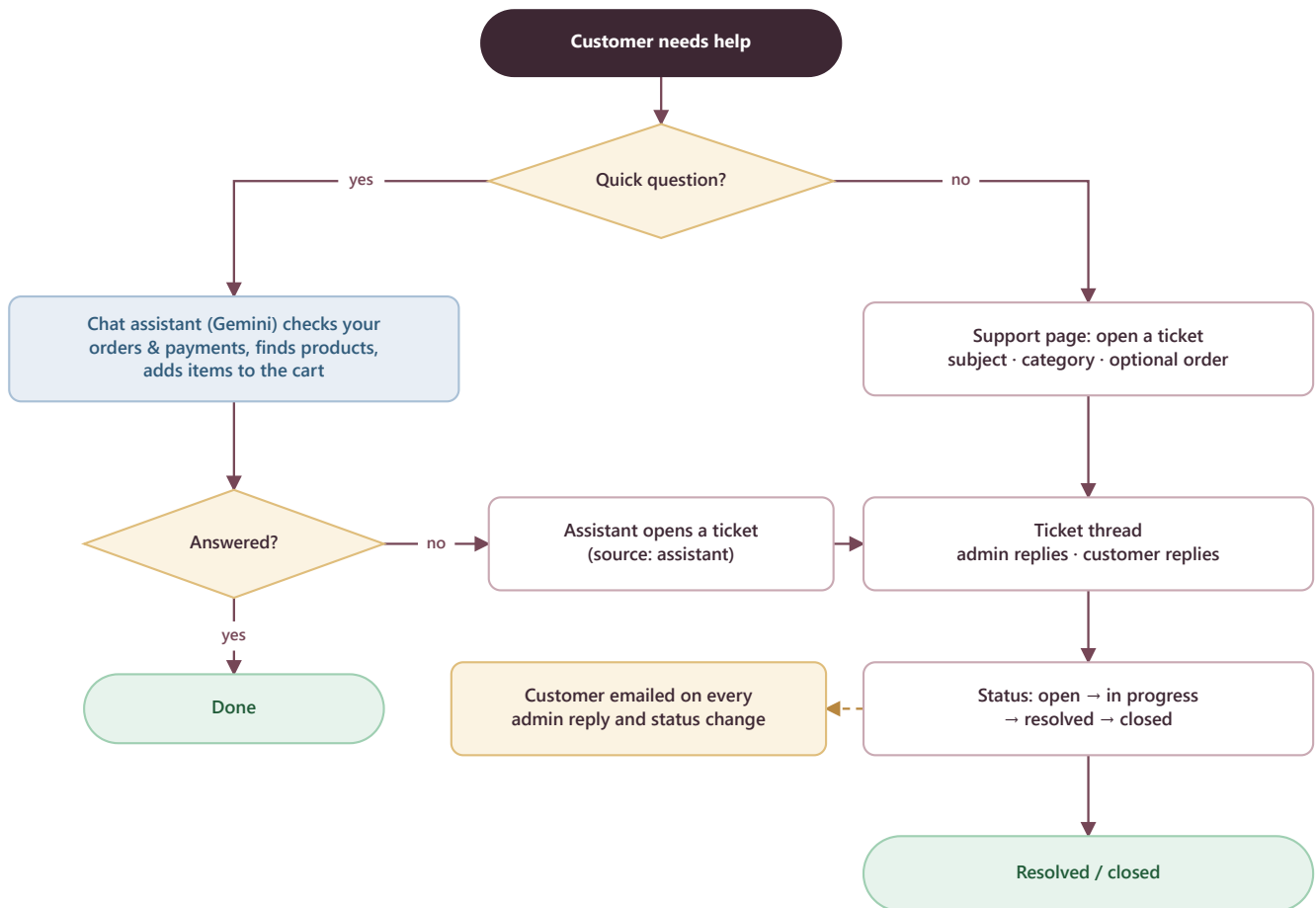


Figure 4.5 — Getting help

- Tickets need a subject (3–150 chars), one of 8 categories and a description (10–4000 chars); an order can be linked. A duplicate within 10 minutes returns the existing ticket; at most 10 can be open at once.
- An admin reply moves **open** to **in_progress**; a customer reply reopens a **resolved** ticket; closed tickets cannot be replied to.
- The assistant can list orders, payments and tickets, look up one order, search products, add items to the cart (with a “Go to checkout” button) and open a ticket. It cannot cancel, refund or change addresses. Conversations hold 30 turns; “New chat” starts again.

4.8 Admin workflow

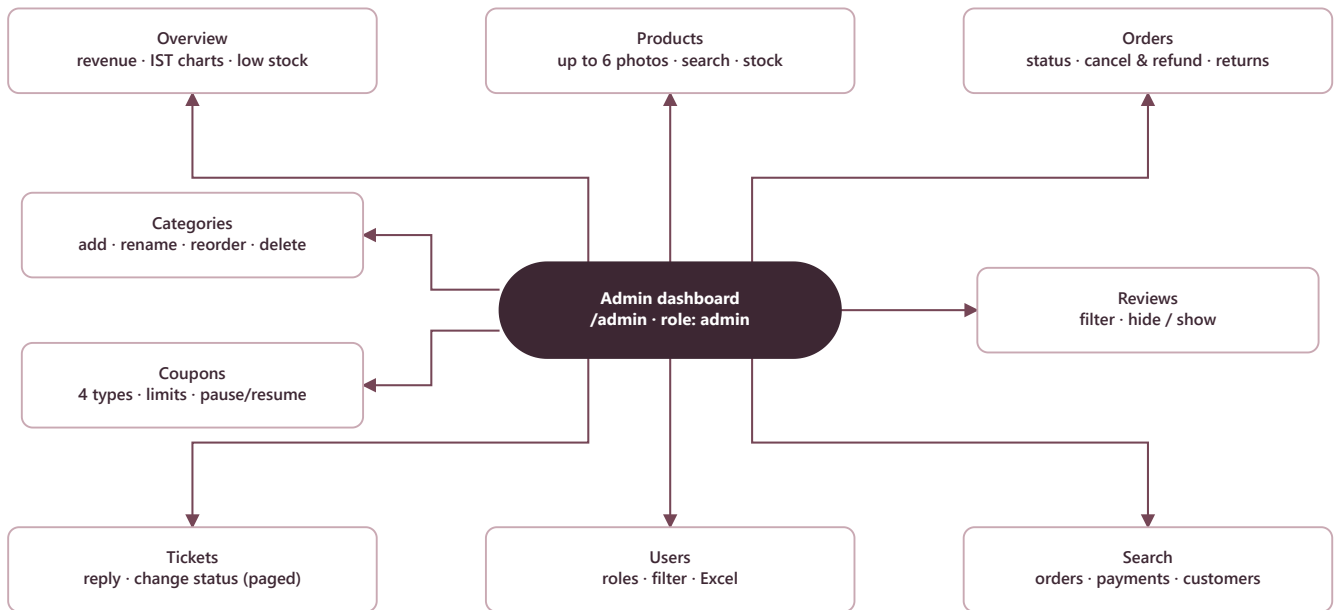


Figure 4.6 — The admin dashboard and its tabs

Tab	What an admin can do
Overview	Customers, orders, products, paid revenue, paid orders, this month, average order value; last 7 days and 6 months of revenue by India calendar day / month; order status split; low stock (≤ 5); top 5 products.
Products	Add or edit name, description, price, stock, category and up to 6 photos (5 MB each; keep, remove or make one the main photo); delete (also deletes its reviews). 50 per page.
Categories	Add, rename, describe, reorder and delete categories. Renaming updates its products and coupons; a category still in use cannot be deleted.
Orders	20 per page with status filters and counts. Set pending → shipped → delivered; cancel an unshipped order or mark a shipped / delivered one returned (choosing whether to restock) — both refund through Razorpay and email the customer. When a cancel or return refund fails, every admin is emailed; a Refund problems filter lists failed refunds and ones interrupted for 10 minutes, each with Retry (which first checks Razorpay, so a refund already made — even by hand in the Razorpay dashboard — is recorded, not repeated). Delete an order record (not while its refund is unfinished).
Coupons	Create / edit every rule (type, value, dates, limits, eligible products from a searchable picker, categories, customers, payment methods, visibility); pause or resume; delete.
Reviews	Filter by visible / hidden and rating; hide or show. Hidden reviews stay with the author but drop out of the page and the rating.
Tickets	Filter by status with counts, 50 per page, reply, change status; the customer is emailed each time. After a change the page reloads and refills; a reply that moves the open ticket to the top is followed to page 1 with its thread still open.
Users	Table with orders, total spent (refunds excluded) and join date; filter; "Download Excel" (.xlsx); make verified users admins or admins customers (not yourself, never the last admin); the person sees the new role after signing in again.
Search	Find orders, payment records and customers by ID, short code (#XXXXXXXX), <code>pay_...</code> , <code>order_...</code> or email; a paid checkout Razorpay is still confirming shows "Paid, being confirmed"; delete records and customer accounts where safe.

API workflows

Sequence diagrams for the main requests. Solid arrows are calls, dashed arrows are replies, numbers give the order. All paths are relative to <https://anm-shop-api.onrender.com>.

5.1 Sign-up and email verification

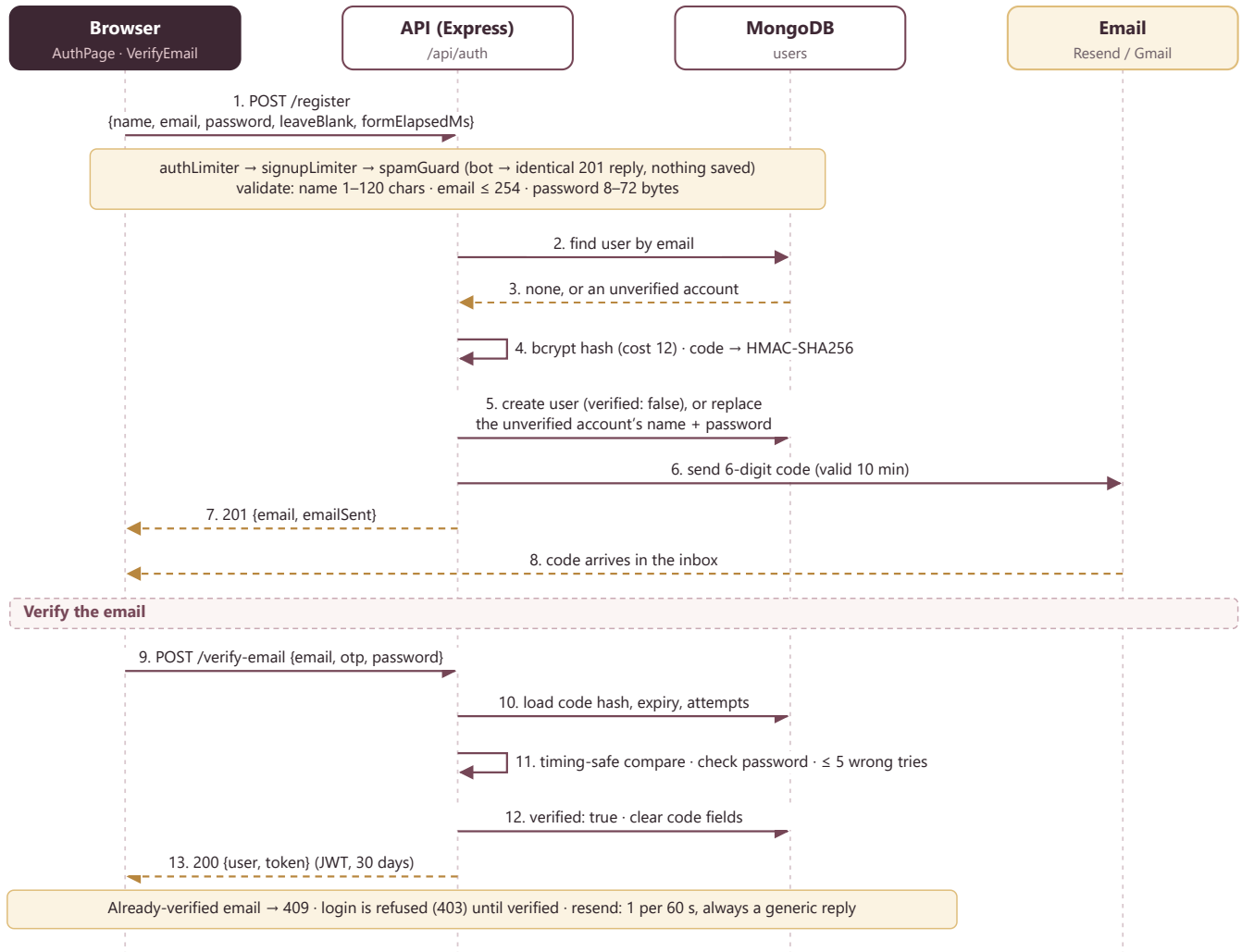


Figure 5.1 — `POST /api/auth/register` and `/verify-email`

5.2 Sign-in and authenticated requests

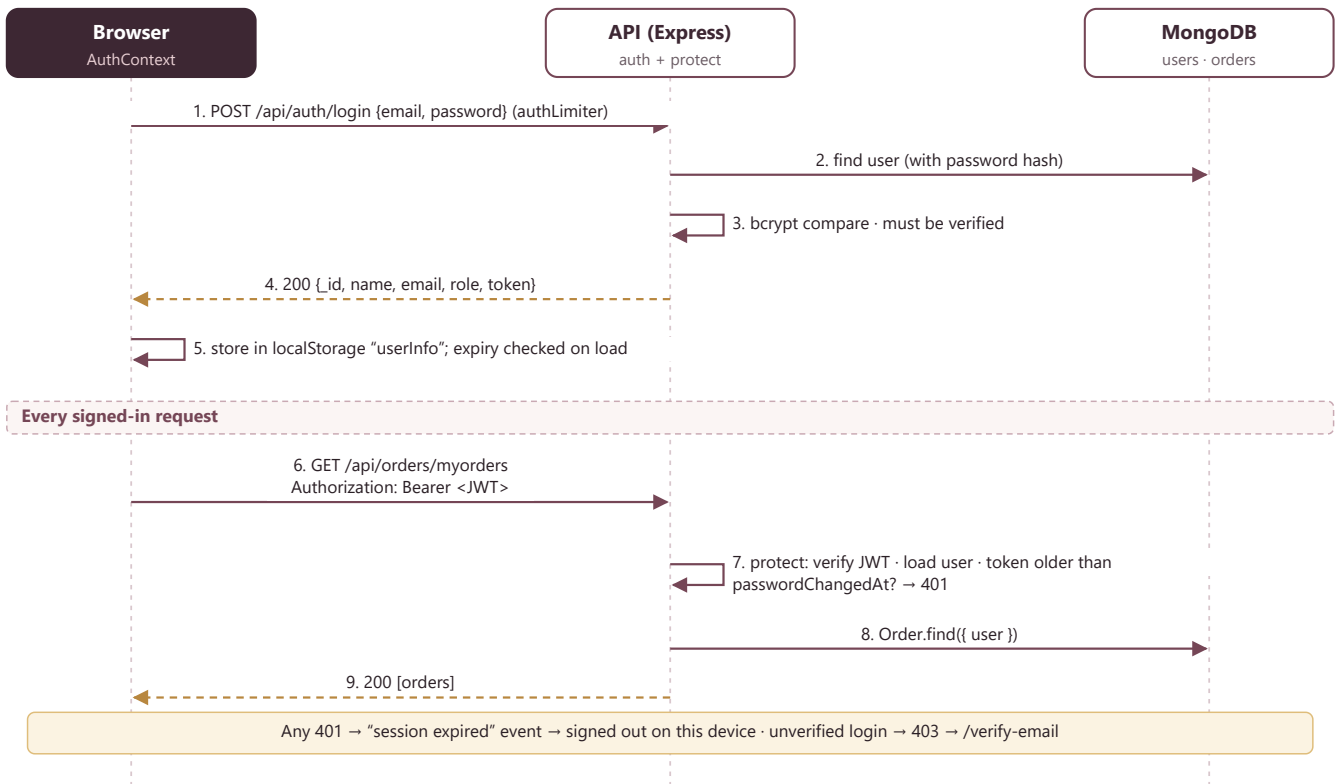


Figure 5.2 — POST /api/auth/login, then any protected route

5.3 Password reset

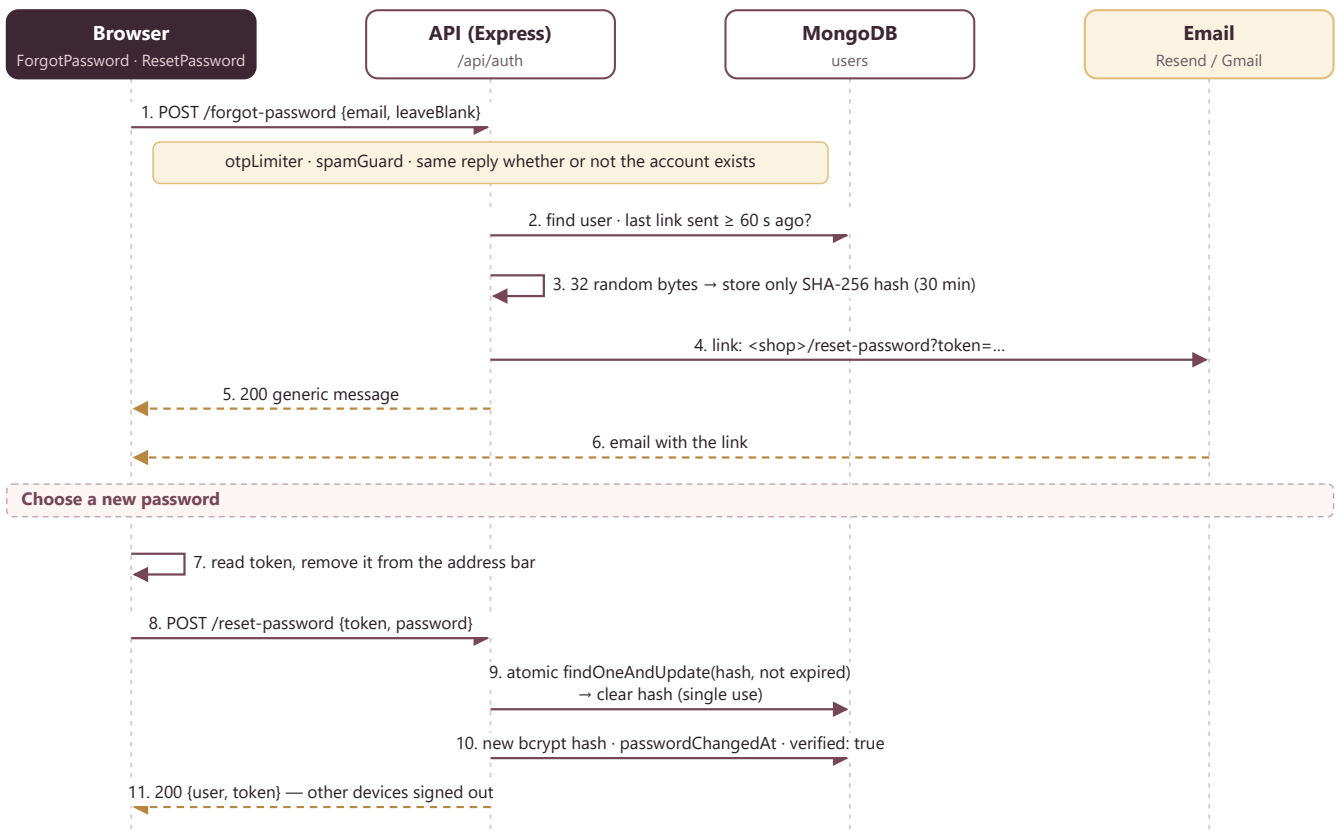


Figure 5.3 — POST /api/auth/forgot-password and /reset-password

5.4 Checkout and payment

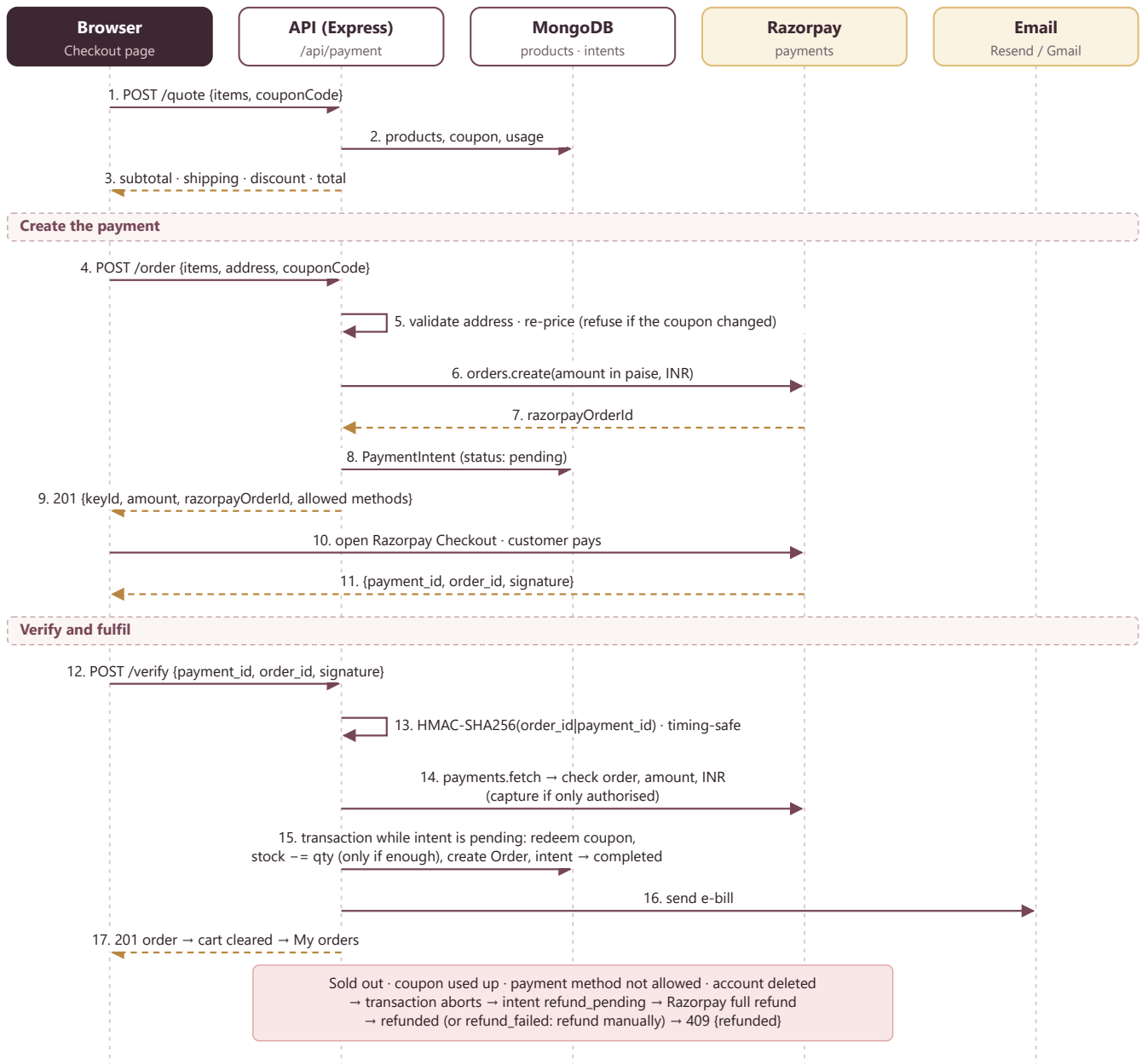


Figure 5.4 — Quote, create the payment, pay in Razorpay, verify and fulfil

5.5 Razorpay webhook (backup path)

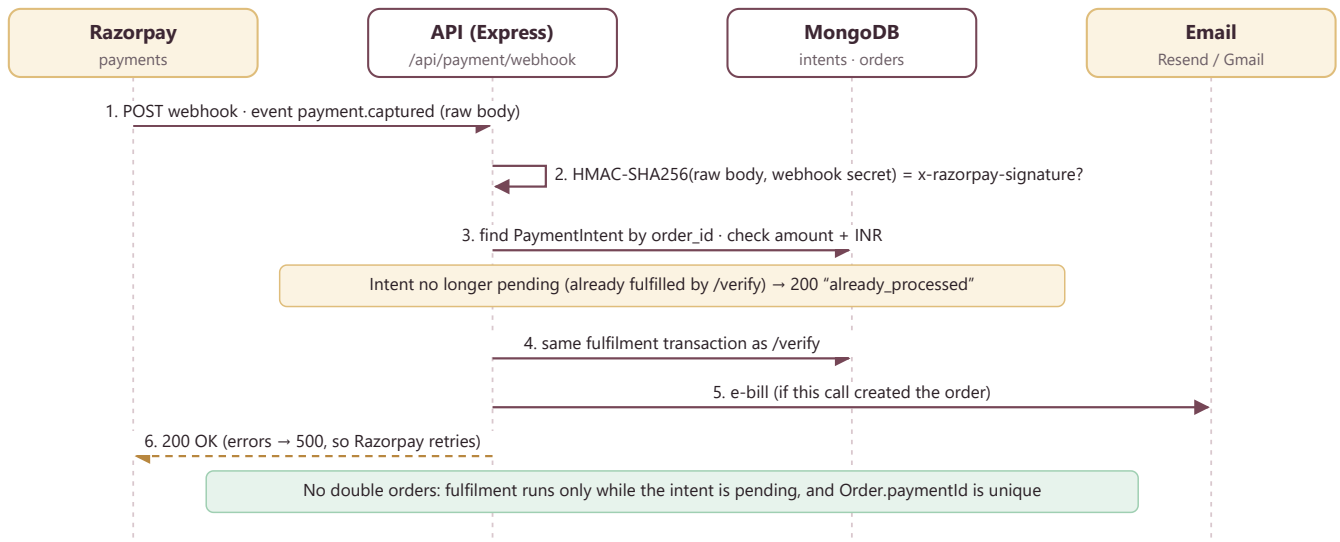


Figure 5.5 — POST `/api/payment/webhook` creates the order if the browser never called `/verify`

Configure the webhook in the Razorpay dashboard: URL `https://<api>/api/payment/webhook`, event `payment.captured`, secret = `RAZORPAY_WEBHOOK_SECRET`. The route is registered before the JSON parser so the signature is checked against the exact raw body.

5.6 Cancel or return an order

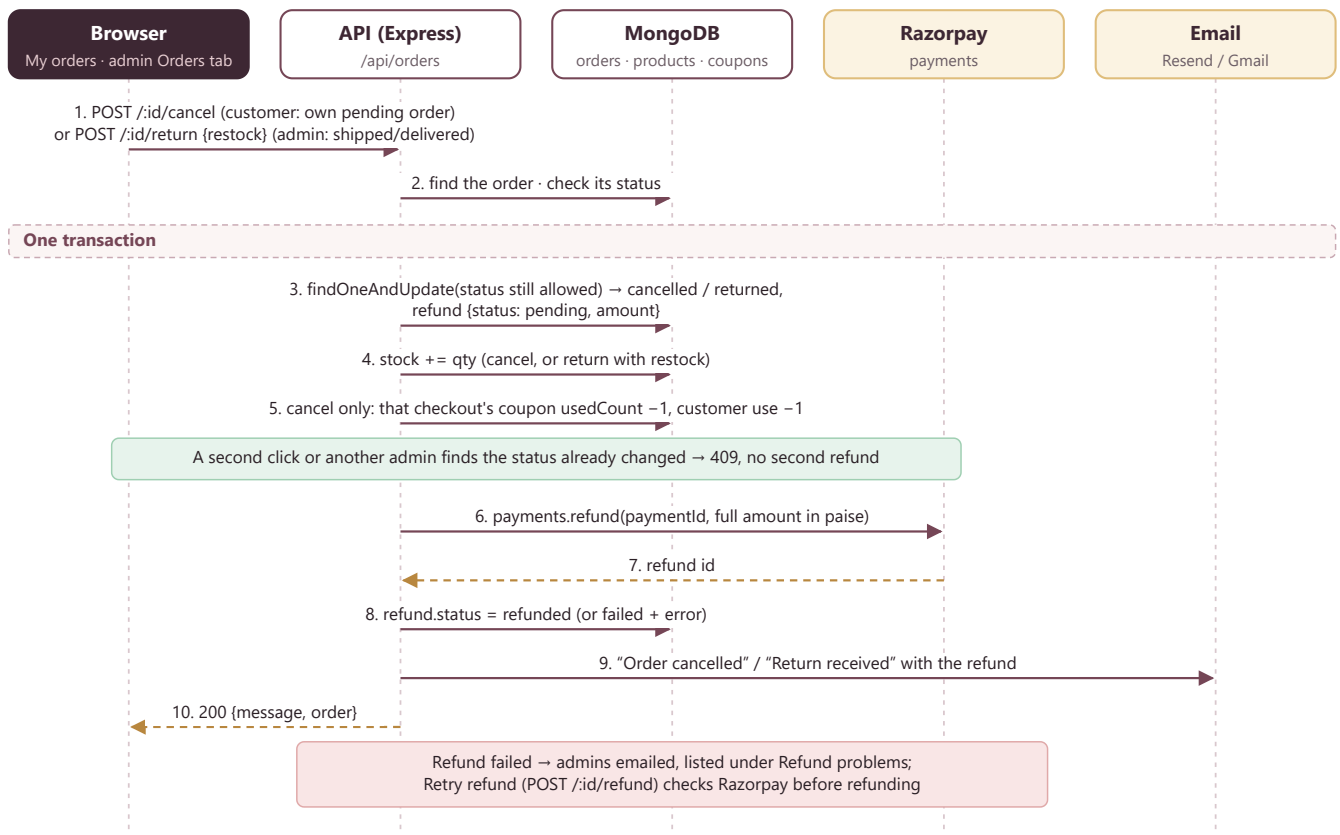


Figure 5.6 — POST `/api/orders/:id/cancel`, `/return` and `/refund`

5.7 AI chat assistant

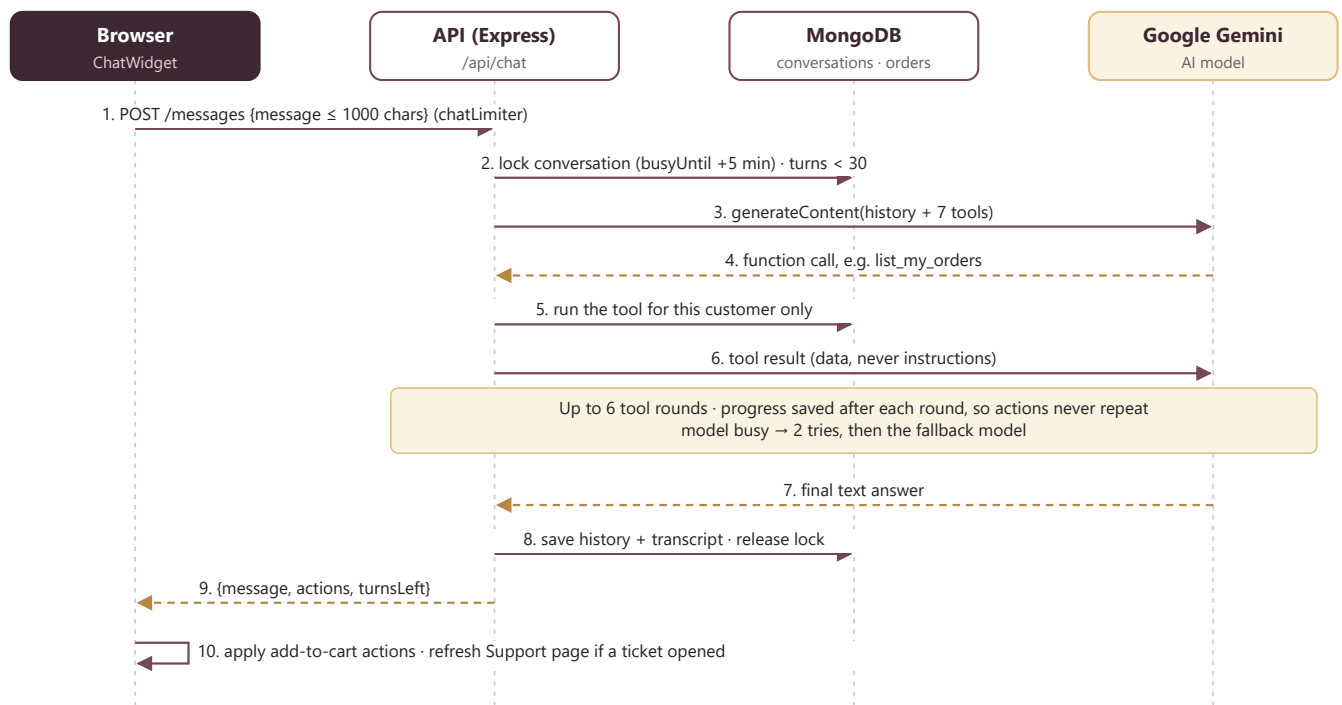


Figure 5.7 — `POST /api/chat/messages` and the tool loop

Low-level design (LLD)

6.1 Backend request pipeline

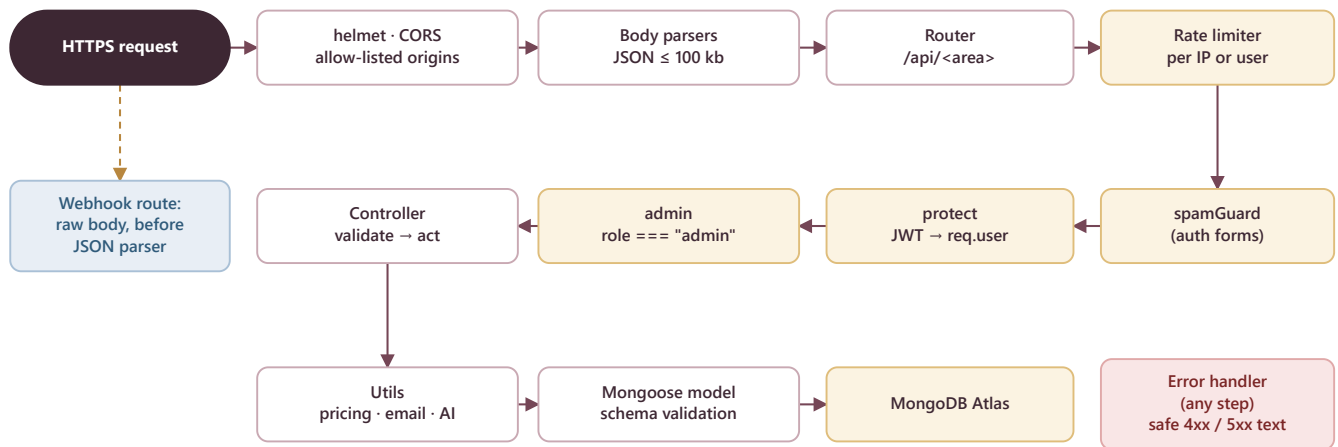


Figure 6.1 — What every API request passes through (gold steps are added per route)

1. `x-powered-by` off; `trust proxy` = `TRUST_PROXY` (1 on Render) so per-IP limits see the real client.
2. `helmet()`, then CORS: no Origin, an origin in `FRONTEND_URL`, or (outside production) localhost on any port.
3. `POST /api/payment/webhook` with a raw body (1 MB) — before any JSON parsing.
4. `express.json` (100 kb), `express.urlencoded` (20 kb, 50 fields), then `req.body ??= {}` because Express 5 leaves it undefined.
5. Routers. Per route: rate limiter → `spamGuard` → `protect` → `admin` → `multer` (images) → controller.
6. Error handler: upload errors → 400, CORS → 403, everything else → a generic message without internal details.

6.2 Backend modules

Module	Responsibility
<code>controller/authController.js</code>	Register, verify, resend code, login, change / forgot / reset password, delete own account, list and delete users (shared deletion checks, one transaction), change roles
<code>app.js</code> · <code>index.js</code>	The Express app (middleware, routes, error handler) · start-up: database, default categories, fix-ups, listen
<code>controller/ProductController.js</code>	Catalogue with search (text-index matches ranked first, then partial-word matches), admin list and search, cart look-up, create / update with up to 6 photos, delete
<code>controller/categoryController.js</code>	Public category list; admin list with usage counts; create, rename (with products and coupons) and delete
<code>controller/reviewController.js</code>	Product reviews, own review, admin moderation
<code>controller/paymentController.js</code>	Quote, create payment, verify, webhook, fulfilment transaction, refunds, payment-record cleanup
<code>controller/orderController.js</code>	My orders, paged admin list, status, cancel, return, retry refund, delete, resend e-bill

Module	Responsibility
<code>controller/couponController.js</code>	Coupon CRUD, pause / resume, "My coupons"
<code>controller/ticketController.js</code>	Create, list, reply, change status
<code>controller/chatController.js</code>	Conversation lock, turn limit, calling the assistant, reset
<code>controller/AnalyticsController.js</code>	Dashboard statistics in Asia/Kolkata time
<code>controller/searchController.js</code>	Admin search by IDs, short codes, Razorpay IDs, email
<code>controller/sitemapController.js</code>	XML sitemap of pages, categories and up to 45,000 products
<code>utils/pricing.js</code>	Cart lines, coupon status and eligibility, discount maths, quote
<code>utils/categories.js</code>	Cached category list, default categories on first start
<code>utils/orderRefunds.js</code> · <code>razorpayClient.js</code>	Cancel / return transaction, Razorpay refund, admin alert, retry (checking Razorpay first), customer email · shared Razorpay client
<code>utils/supportAssistant.js</code>	Gemini calls, retries, fallback model, 7 tools, system prompt
<code>utils/sendEmail.js</code> · <code>sendOrderInvoice.js</code>	Resend or Gmail delivery; HTML + text e-bill (India time; the seller from <code>config/business.js</code>)
<code>utils/tickets.js</code> · <code>orderLookup.js</code> · <code>reviews.js</code>	Shared ticket creation, #short codes, rating recount and legacy review migration
<code>utils/orderMigrations.js</code>	One-time order breakdown fix, recorded in <code>migrations</code>

6.3 Frontend architecture

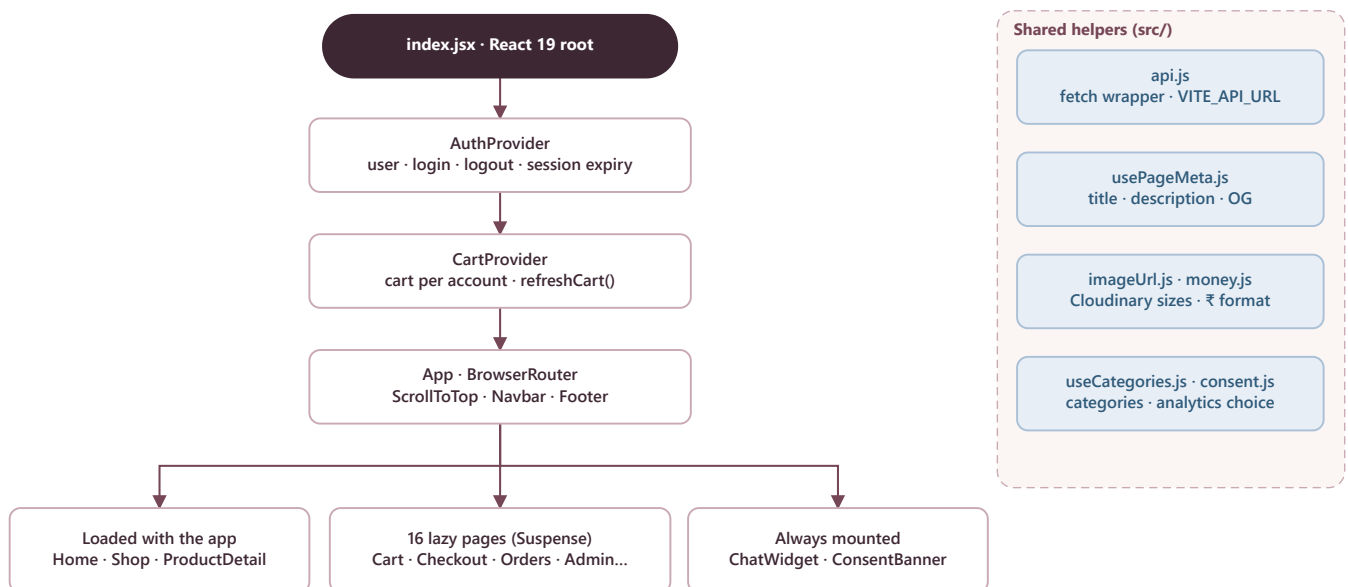


Figure 6.2 — Component tree and shared helpers

- **AuthProvider** holds `{user, login, logout}`, persists the user in `localStorage` and listens for the session-expired event.
- **CartProvider** holds `{cart, itemCount, addToCart, updateQuantity, removeFromCart, clearCart, refreshCart}`. Carts are stored per account (`shopnestCart:<userId>`, `shopnestCart:guest`); the guest cart merges on sign-in.

- **api.js** wraps `fetch`: base URL from `VITE_API_URL` (default `/api`), errors carry `status` and `data`, a network failure becomes status 0.
- **usePageMeta** sets the title, description, canonical URL, Open Graph and Twitter tags per page, and `noindex` on private pages.
- **useCategories** is a small shared store (`useSyncExternalStore`): the categories load once for every page; the admin Categories tab, or an old category link, refreshes them, and only the newest answer is used.
- **Page loading**: the footer sits inside the same `Suspense` boundary as the pages and loading placeholders fill the screen, so the footer no longer jumps down while a page arrives.
- **Tests**: `npm test` in `frontend/` renders the admin Orders, Tickets and Search tabs in jsdom against a fake API that follows the server's rules, the Contact and policy pages with sample business details, and the PDF bill with jsPDF faked.
- **Business details**: `src/data/contactInfo.js` holds the legal name, address, phone and GSTIN, and the grievance officer. They ship empty; each part appears on the Contact page, footer, Terms, Privacy policy and PDF bill only once it is filled in.

6.4 Data model

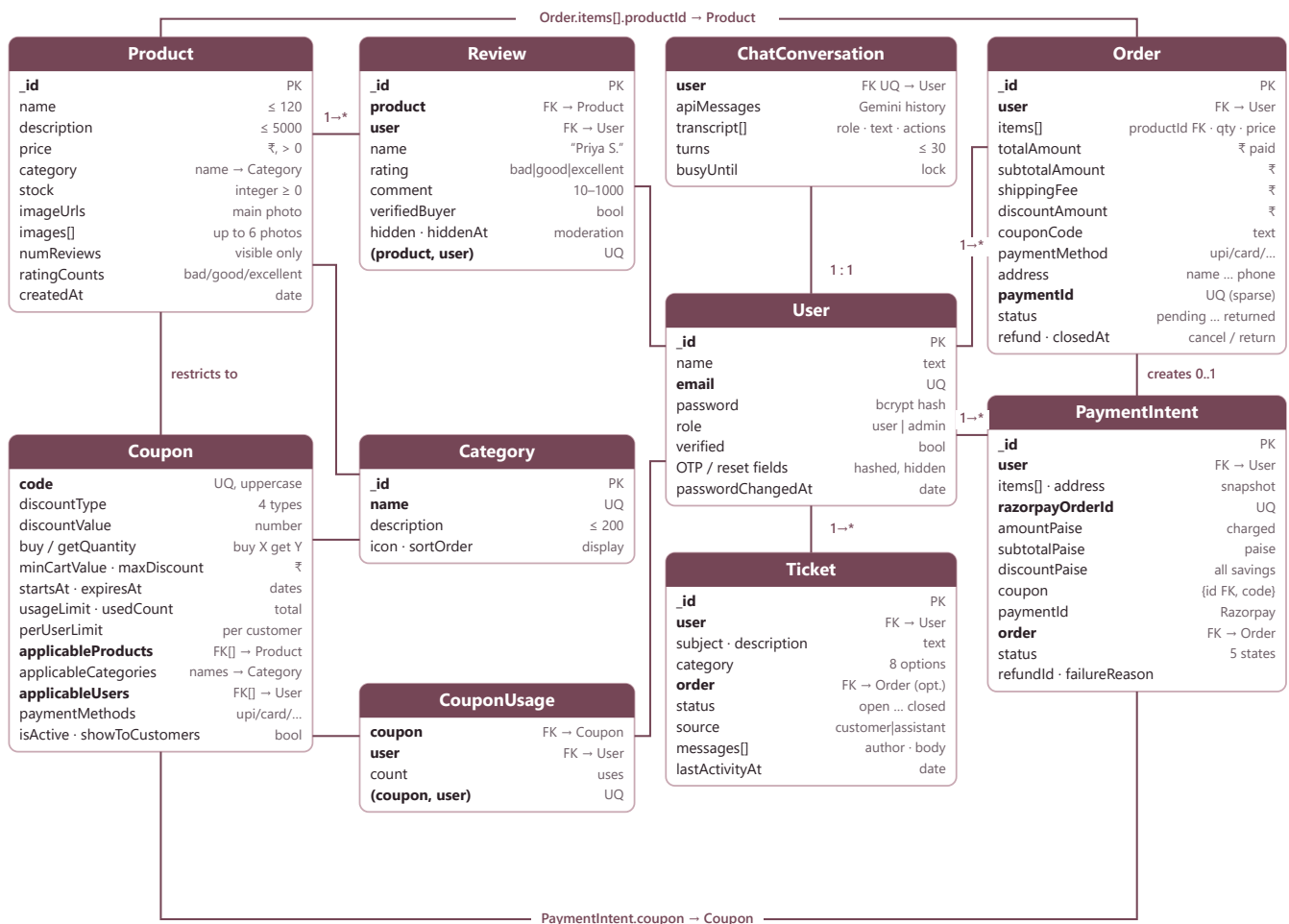


Figure 6.3 — Collections, key fields and relationships (PK primary key, FK reference, UQ unique)

- A **User** has many Orders, PaymentIntents, Reviews, Tickets and CouponUsages, and at most one ChatConversation.
- A **PaymentIntent** is one checkout attempt. It snapshots items, address and amounts, and links to the **Order** it created. A `pending` one with a `paymentId` was paid and is waiting for Razorpay to confirm it.
- **Product.numReviews** and **ratingCounts** are denormalised from visible Reviews and recounted on every change.
- **CouponUsage** counts uses per customer per coupon (unique pair) for `perUserLimit`.

- **Category** names are stored on products and in coupon restrictions, so a rename updates both in one transaction. Products in a category that no longer exists are hidden from the shop and cannot be bought.

6.5 Pricing engine

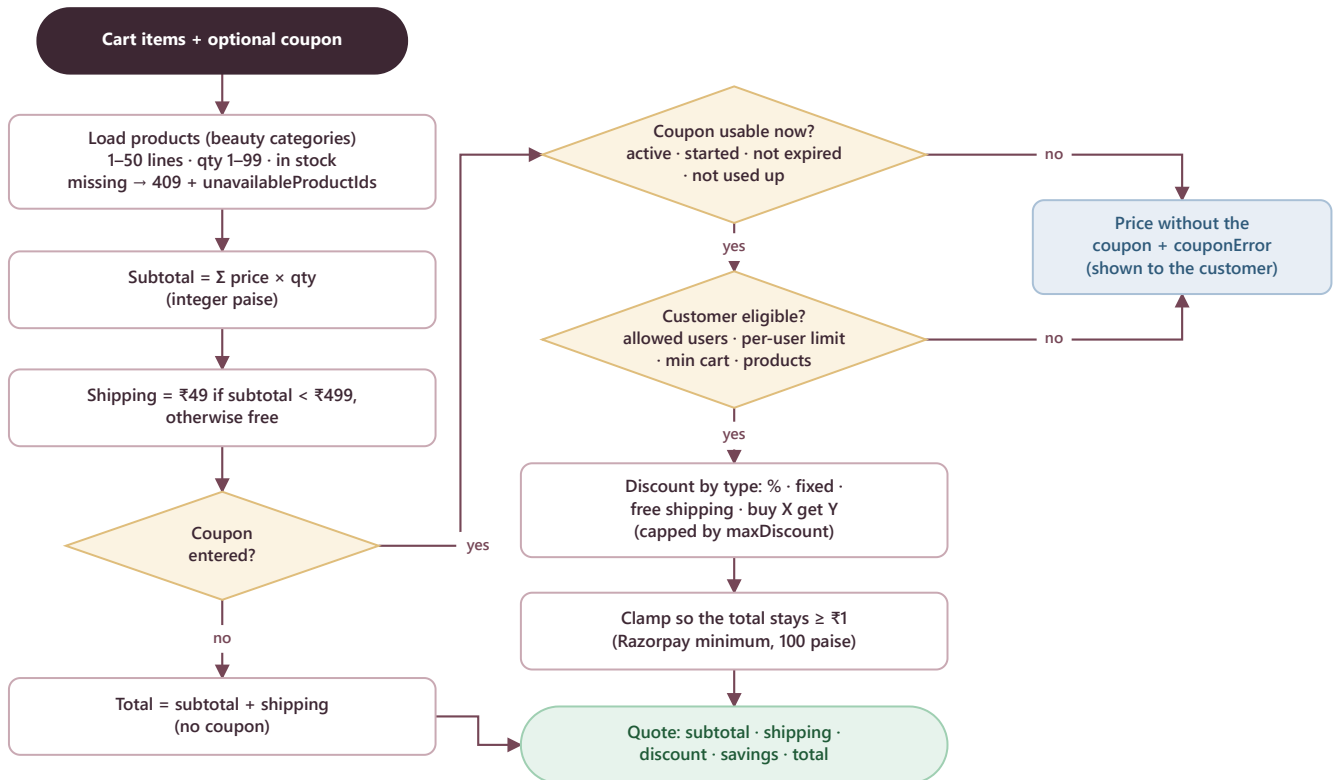


Figure 6.4 — `utils/pricing.js`: from cart to quote

Coupon type	Discount	Notes
percentage	1–100 % of the eligible subtotal (rounded down)	Optional <code>maxDiscount</code> cap
fixed	₹1 – ₹10,000,000, at most the eligible subtotal	<code>maxDiscount</code> not used
free_shipping	Waives the ₹49 fee	Fails with “This order already ships free” when shipping is already free
buy_x_get_y	Units sorted by price (highest first); in each full group of X+Y, the Y cheapest are free	X and Y 1–20 each; optional cap

Eligible lines are those matching `applicableProducts` or `applicableCategories` (none set = all). `minCartValue` is checked against the whole subtotal. The discount is clamped so the total stays at least ₹1, Razorpay’s minimum.

6.6 State machines

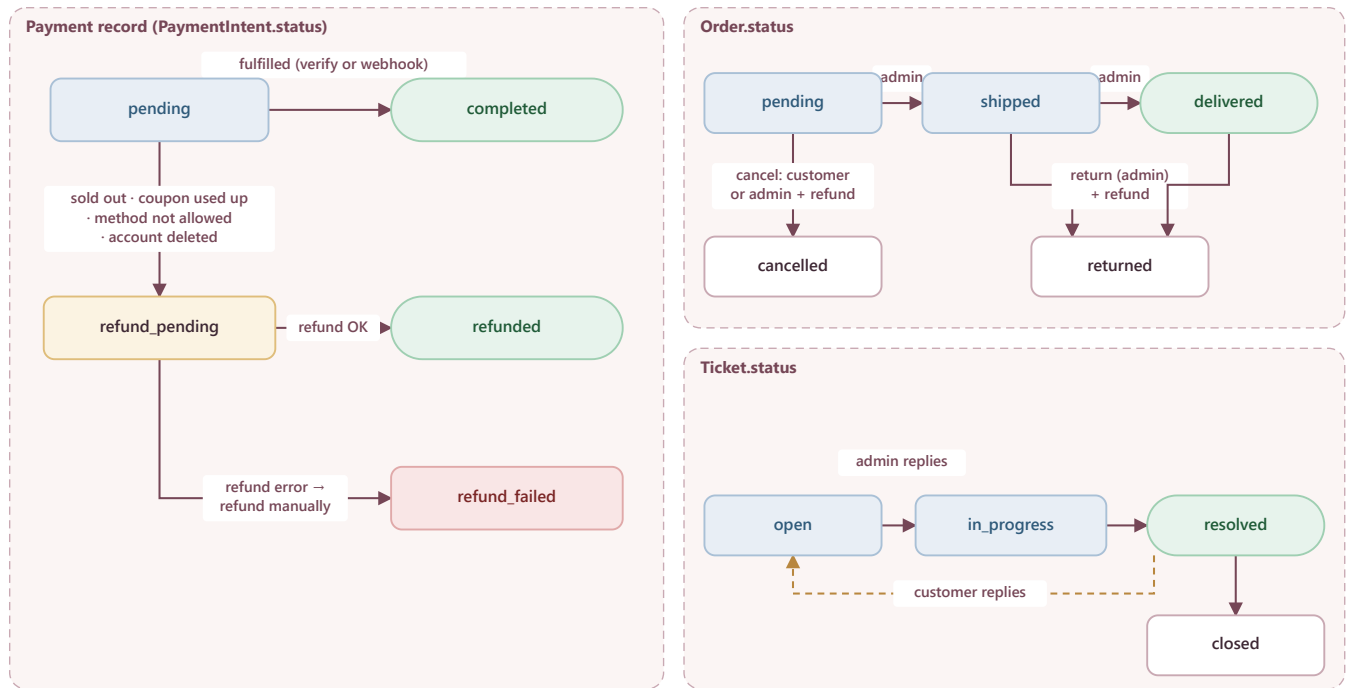


Figure 6.5 — Status lifecycles for payment records, orders and tickets

Coupon status	When
disabled	<code>isActive</code> is false (paused)
scheduled	<code>startsAt</code> is in the future
expired	<code>expiresAt</code> has passed
used_up	<code>usedCount</code> has reached <code>usageLimit</code>
active	none of the above

6.7 Payment fulfilment internals

- Create:** validate the address, re-price, create a Razorpay order (amount in paise, INR, random receipt) and a `PaymentIntent` holding items, address, `amountPaise`, `subtotalPaise`, `discountPaise` (every saving including waived shipping), coupon and allowed methods.
- Verify:** find the intent for this user; check the HMAC-SHA256 signature of `order_id|payment_id` with a timing-safe compare; fetch the payment and check order, amount and currency; capture if only authorised (still authorised → the payment ID is noted on the record and 202 "pending"; the webhook finishes it).
- Fulfil (transaction):** refuse if the account has been deleted meanwhile (the payment is refunded instead); redeem the coupon (method allowed, `usedCount` < limit, per-user count < limit); for each item `stock -= qty` only where `stock ≥ qty`; create the Order with shipping = (amount – subtotal + discount) / 100; mark the intent `completed`.
- Refuse → refund:** any failed step aborts the transaction. The intent is claimed atomically (`pending` → `refund_pending`), a full refund is requested, and the status becomes `refunded` or `refund_failed` (logged as AUTOMATIC REFUND FAILED for a manual refund).
- Notify:** on success the e-bill is emailed and `invoiceEmailSent` is recorded.

6.8 Cancellations and returns

- A cancel (customer: own pending order; admin: any pending order) or return (admin: shipped or delivered) runs one transaction: a `findOneAndUpdate` that only matches while the status still allows it sets the new status and `refund: {status: pending, amount}`, puts stock back (always for cancels, by choice for returns) and, for cancels, gives back the coupon use to the exact coupon named on the checkout record.
- The full amount is then refunded through Razorpay (the reason goes in a note, cut to 250 characters, within Razorpay's 256-character limit); the order records `refunded` with the refund ID, or `failed` with Razorpay's error. When that first attempt fails, every admin is emailed in the background. Once the order is closed, a failing email (to the admins or the customer) or a failure to record the refund's result no longer makes the request fail.
- Retry works on failed refunds and on ones still pending after 10 minutes. It first lists the payment's refunds at Razorpay: one noted with this order, or refunds that together cover the amount (e.g. made by hand in the dashboard), is recorded instead of refunding again.
- Statuses `cancelled` and `returned` are final: the manual status change only applies while an order is pending, shipped or delivered, in the same atomic step. Revenue figures leave them out, the customer's "verified buyer" label goes once every order of that product is cancelled, and an order with an unfinished refund cannot be deleted.

6.9 AI assistant internals

Tool	What it returns or does
<code>list_my_orders</code>	The customer's last 10 orders
<code>get_order(order_code)</code>	One order with its payment record
<code>list_my_payments</code>	Last 10 payment records with readable status (including "paid but being confirmed")
<code>search_products(query?, category?)</code>	Up to 8 products with review counts
<code>add_to_cart(product_id, quantity 1-10)</code>	An action the browser applies (capped at stock), plus a checkout button
<code>create_support_ticket(...)</code>	Opens a ticket with <code>source: assistant</code>
<code>list_my_tickets</code>	Tickets with the latest team reply

Limits: 1000 characters per message, 30 turns per chat, 6 tool rounds per message, one message at a time (5-minute lock), 60-second model timeout, 2 tries per model on 5xx before the fallback. Quota errors return "usage limit", a bad key switches the widget to offline mode, and without `GEMINI_API_KEY` the widget points customers to tickets.

API reference

60 endpoints. `public` needs nothing, `signed in` needs a Bearer JWT, `admin` also needs `role: admin`; gold tags are rate limiters (Part 10).

Root

	Path	Access	Purpose
GET	/	public	Health check <code>{service, status: "ok"}</code> (used by Render)
GET	/sitemap.xml	public	XML sitemap; Vercel proxies the storefront's /sitemap.xml here
POST	/api/payment/webhook	public	Razorpay <code>payment.captured</code> webhook; raw body, HMAC-checked

/api/auth

	Path	Access	Purpose
POST	/api/auth/register	public <code>authLimiter</code> <code>signupLimiter</code> <code>spamGuard</code>	Create an account (or replace an unverified one) and email a 6-digit code
POST	/api/auth/verify-email	public <code>otpLimiter</code>	Email + code + password → verified; returns user and JWT
POST	/api/auth/resend-verification	public <code>otpLimiter</code> <code>spamGuard</code>	New code (generic reply, 60 s gap)
POST	/api/auth/login	public <code>authLimiter</code>	Sign in; 403 <code>verificationRequired</code> if unverified
PUT	/api/auth/password	signed in <code>authLimiter</code>	Change password; signs out other sessions; new token
POST	/api/auth/forgot-password	public <code>otpLimiter</code> <code>spamGuard</code>	Email a single-use reset link (30 min); generic reply
POST	/api/auth/reset-password	public <code>otpLimiter</code>	New password from the token; verifies; returns user and JWT
GET	/api/auth/users	admin	All accounts with orders, total spent, last order, joined
DELETE	/api/auth/users/:id	admin	Delete a non-admin account, its chat and reviews (orders, payments and tickets kept); same checks as deleting your own
PUT	/api/auth/users/:id/role	admin	<code>{role: user admin}</code> for another verified account; never your own, never the last admin

	Path	Access	Purpose
DELETE	/api/auth/me	signed in authLimiter	Delete your own account (password required; not while an order is on its way, a refund is owed or a payment is being confirmed; not admins)

/api/products

	Path	Access	Purpose
GET	/api/products	public	Catalogue: <code>page</code> , <code>limit</code> (24, max 100), <code>search</code> (whole words by relevance, then partial words), <code>category</code> (unknown → 400 <code>unknownCategory</code>)
POST	/api/products	admin	Create (multipart <code>images</code> : 1–6 photos, 5 MB each, JPEG/PNG/WebP)
GET	/api/products/manage	admin	Admin list (all categories, paginated, optional <code>search</code>)
GET	/api/products/lookup	public lookupLimiter	?ids=a,b,c (≤ 50): current price and stock for cart refresh
GET	/api/products/:id/reviews	public	Visible reviews (10 per page, max 50) + summary
POST	/api/products/:id/reviews	signed in reviewLimiter	Create or edit your review
GET	/api/products/:id/reviews/mine	signed in	Your review, including whether it is hidden
DELETE	/api/products/:id/reviews/mine	signed in reviewLimiter	Delete your review
GET	/api/products/:id	public	One product (404 if its category is no longer one of the shop's)
PUT	/api/products/:id	admin	Partial update; <code>keepImages</code> (JSON list) + new <code>images</code> , up to 6 in total
DELETE	/api/products/:id	admin	Delete the product and its reviews

/api/payment

	Path	Access	Purpose
POST	/api/payment/quote	signed in quoteLimiter	Server price: subtotal, shipping, discount, savings, total, couponError
POST	/api/payment/order	signed in paymentLimiter	Validate address, re-price, create Razorpay order + PaymentIntent
POST	/api/payment/verify	signed in paymentLimiter	Check signature and payment, capture, fulfil, send e-bill
DELETE	/api/payment/records/:id	admin	Delete a payment record (not recent pending or refund_pending)

/api/orders

	Path	Access	Purpose
GET	/api/orders	admin	All orders, 20 per page, optional <code>status</code> or <code>refund=problem</code> , with per-status counts and <code>refundProblems</code>
GET	/api/orders/myorders	signed in	Your orders
POST	/api/orders/:id/resent-invoice	signed in invoiceEmailLimiter	Email the e-bill again (your paid order; not cancelled or returned)
PUT	/api/orders/:id/status	admin	Set pending, shipped or delivered (only while it is one of those)
POST	/api/orders/:id/cancel	signed in	Cancel a pending order (customers: their own; admins: any) — full refund, stock back, coupon use back
POST	/api/orders/:id/return	admin	Mark a shipped / delivered order returned <code>{restock, reason}</code> — full refund
POST	/api/orders/:id/refund	admin	Retry a failed (or stuck) order refund; records a refund Razorpay already made instead of repeating it
DELETE	/api/orders/:id	admin	Delete the order record only (no refund, no stock restore); refused while its refund is pending or failed

/api/coupons

	Path	Access	Purpose
GET	/api/coupons/mine	signed in	Coupons this customer can use, with uses left
GET	/api/coupons	admin	All coupons with status and summary
POST	/api/coupons	admin	Create a coupon
PUT	/api/coupons/:id	admin	Replace its settings (usedCount untouched)
PATCH	/api/coupons/:id/active	admin	Pause or resume <code>{isActive}</code>
DELETE	/api/coupons/:id	admin	Delete the coupon and its usage counts

/api/reviews (moderation)

	Path	Access	Purpose
GET	/api/reviews	admin	All reviews; filter by status and rating; counts
PATCH	/api/reviews/:id	admin	Hide or show <code>{hidden}</code> ; recounts the rating

/api/tickets

	Path	Access	Purpose
POST	/api/tickets	signed in ticketLimiter	Open a ticket
GET	/api/tickets/mine	signed in	Your tickets (up to 50)
POST	/api/tickets/:id/messages	signed in ticketReplyLimiter	Reply (own ticket, or any ticket as admin; admin replies email the customer and return the order's details)
GET	/api/tickets	admin	All tickets, 50 per page (?status= , ?page=), with counts
PUT	/api/tickets/:id/status	admin	Change status and email the customer

/api/categories

	Path	Access	Purpose
GET	/api/categories	public	Categories in display order (name, description, icon)
GET	/api/categories/manage	admin	Every category with how many products and coupons use it
POST	/api/categories	admin	Add a category (name 2–60, unique ignoring case)
PUT	/api/categories/:id	admin	Edit; a rename also renames it on products and coupons
DELETE	/api/categories/:id	admin	Delete an unused category (409 while products or coupons use it)

/api/chat • /api/analytics • /api/admin/search

	Path	Access	Purpose
GET	/api/chat	signed in	Transcript, configured flag, turns left
POST	/api/chat/messages	signed in chatLimiter	Send a message to the assistant; returns reply and actions
DELETE	/api/chat	signed in	Start a new chat
GET	/api/analytics	admin	Dashboard statistics (India time)
GET	/api/admin/search	admin	?q= order / payment / user ID, #code, pay_..., order_..., or email

Data model reference

User

Field	Type and rules
name	String, required
email	String, required, unique
password	bcrypt hash, required
role	<code>user</code> <code>admin</code>
verified	Boolean, default false
verificationOtp*	hash, expiry, sent-at, attempts — hidden
passwordReset*	token hash, expiry, sent-at — hidden
passwordChangedAt	Date (revokes older tokens)

No timestamps; “joined” comes from the ObjectId.

Product

Field	Type and rules
name	≤ 120 chars
description	≤ 5000 chars
price	0.01 – 100,000,000
category	a Category name (checked in controllers)
stock	integer 0 – 1,000,000
imageUrls	main photo URL
images[]	up to 6 photo URLs
numReviews · ratingCounts	visible reviews only
createdAt	Date, default now
text index	name (×5) + description

Order

Field	Type and rules
user	→ User
items[]	productId → Product, qty ≥ 1, price
totalAmount	₹ paid
subtotalAmount · shippingFee · discountAmount	₹ breakdown
couponCode · paymentMethod	String
address	fullName, street, city, postalCode, country, phone
paymentId	unique, sparse
invoiceEmailSent	Boolean
status	pending shipped delivered cancelled returned
closedAt · closedBy · restocked	when / who cancelled or returned it
refund	status, amount, Razorpay refund ID, reason, error
timestamps	createdAt, updatedAt

PaymentIntent

Field	Type and rules
user	→ User
items[] · address	snapshot at checkout
razorpayOrderId	unique
amountPaise	amount charged
subtotalPaise · discountPaise	paise; discount = all savings
coupon	{id → Coupon, code}
allowedPaymentMethods	String[]
paymentId · order	Razorpay ID (set once paid) · → Order
status	pending completed refund_pending refunded refund_failed
refundId · failureReason	String

Coupon

Field	Type and rules
code	unique, uppercase, ≤ 30
description	≤ 200
discountType	percentage fixed free_shipping buy_x_get_y
discountValue · buyQuantity · getQuantity	Numbers
minCartValue · maxDiscount	₹
startsAt · expiresAt	Dates
usageLimit · perUserLimit · usedCount	limits
applicableProducts / Categories / Users	empty = no restriction
paymentMethods	upi card netbanking wallet
isActive · showToCustomers	Boolean

Review

Field	Type and rules
product · user	→ Product, → User; unique pair
name	public name, ≤ 60
rating	bad good excellent
comment	≤ 1000 (10+ required in the API)
verifiedBuyer	Boolean
hidden · hiddenAt	moderation
indexes	(product, hidden, createdAt), (createdAt)

Ticket

Field	Type and rules
user · order	→ User, → Order (optional)
subject · description	≤ 150 · ≤ 4000
category	order, payment, refund, delivery, return, product, account, other
status	open in_progress resolved closed
source	customer assistant
messages[]	author, authorName, body ≤ 4000, createdAt
lastActivityAt	Date, indexed

Category

Field	Type and rules
name	unique, 2–60
description	≤ 200
icon	1–2 characters
sortOrder	0–999, display order
timestamps	createdAt, updatedAt

The 7 launch categories are added on first start.

ChatConversation · CouponUsage

Field	Type and rules
ChatConversation.user	→ User, unique
apiMessages	exact Gemini history (JSON)
transcript[]	role, text, actions, at
turns · busyUntil	turn count · lock
CouponUsage.coupon · user	→ Coupon, → User; unique pair
CouponUsage.count	uses so far

Security

Area	Measures
Transport & headers	HTTPS only (production refuses non-HTTPS origins). API: helmet defaults. Storefront: <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy: strict-origin-when-cross-origin</code> , <code>X-Frame-Options: DENY</code> , a full Content-Security-Policy (<code>default-src 'self'</code> ; scripts only from the shop, Razorpay and Vercel analytics, the inline theme script by SHA-256 hash; images from Cloudinary and placeholder.co; API and Razorpay connections; <code>frame-ancestors 'none'</code>), Permissions-Policy blocking camera, microphone and location.
CORS	Only origins listed in <code>FRONTEND_URL</code> (plus localhost outside production); others get 403.
Start-up checks	Production refuses to start with a <code>JWT_SECRET</code> under 32 characters or a placeholder, or without valid HTTPS <code>FRONTEND_URL</code> origins.
Passwords	bcrypt cost 12; 8–72 bytes; never returned (<code>select: false</code>).
Sessions	JWT <code>{id}</code> , 30 days; tokens issued before <code>passwordChangedAt</code> are rejected; deleted users rejected.
Email codes	6 digits from <code>crypto.randomInt</code> , stored as HMAC-SHA256, 10 minutes, 60 s resend gap, 5 wrong tries, timing-safe compare, password also required.
Reset links	32 random bytes, only the SHA-256 hash stored, 30 minutes, single use (atomic), generic replies, token removed from the URL and from analytics.
Bots	Hidden <code>leaveBlank</code> field on sign-up, forgot-password and resend; sign-up also rejects forms sent in under 2 s; bots get a fake success.
Payments	Prices from the database only; HMAC-SHA256 checks for checkout and webhook signatures; payment fetched and matched (order, amount, INR); transactional, idempotent fulfilment; atomic refund claim.
Input	ObjectId checks, regex-escaped search, length limits, address / PIN / phone rules, coupon dates must include a time zone, body size limits, image type and size limits, HTML-escaped emails, Excel cells written as plain values.
AI assistant	Only the signed-in customer's records; tool results treated as data; validated tool inputs; one message at a time; no refunds, cancellations or address changes.
Accounts & roles	Deleting your own account needs the password (a wrong one returns 400, not 401) and waits while money or an order is still in flight; a payment that arrives after deletion is refunded, not turned into an order; coupons meant only for that customer are switched off so they cannot become open to everyone. Role changes: admins only, never their own, only verified accounts become admins, and the last admin cannot be removed.
Refunds	Cancels and returns claim the order atomically in a transaction, so an order is never refunded twice; Razorpay is called once per claim, and a retry first checks Razorpay for a refund already made. A manual status change cannot reopen a cancelled or returned order.
Admin safety	Admins cannot delete admin accounts or themselves, or delete a customer with money or an order in flight; recent pending and in-refund payment records, and orders with an unfinished refund, cannot be deleted.

Privacy. Analytics are cookie-free and load only after consent (`anmShopConsent`). The policy covers India's DPDP Act 2023, names every processor (Razorpay, MongoDB Atlas, Render, Vercel, Cloudinary, Resend / Gmail, Gemini) and discloses that the free Gemini tier may use chat content.

Rate limits

All limiters use express-rate-limit with standard `RateLimit` headers and a JSON message. Counts are kept in memory, so they reset when the API restarts and are not shared between instances.

Limiter	Limit	Window	Key	Applied to
authLimiter	20	15 min	IP	register, login, change password, delete own account
otpLimiter	8	15 min	IP	verify-email, resend, forgot-password, reset-password
signupLimiter	30	60 min	IP	register — counts successes and 409 “already exists”; form errors and 5xx do not count
paymentLimiter	12	15 min	IP	create payment, verify
quoteLimiter	120	15 min	user	price quote
lookupLimiter	1500	15 min	IP	cart look-up (high because mobile networks share IPs)
invoiceEmailLimiter	5	60 min	user	resend e-bill
chatLimiter	30	15 min	user	chat messages
ticketLimiter	5	60 min	user	new tickets
ticketReplyLimiter	30	15 min	user	ticket replies
reviewLimiter	20	60 min	user	save / delete review

Frontend reference

Path	Page	Purpose
/	Home	Hero, a tile per category, feature cards
/shop	Shop	Grid, search, category filter, pages of 24; an old category link shows the whole shop
/product/:id	ProductDetail	Photo gallery, details, add to cart, reviews
/cart	Cart (lazy)	Quantities, refreshed prices, free-shipping hint
/checkout	Checkout (lazy)	Address, live quote, coupons, Razorpay
/login · /register	AuthPage (lazy)	Sign in / sign up (honeypot, timing)
/verify-email	VerifyEmail (lazy)	Code + password, resend
/forgot-password · /reset-password	ForgotPassword · ResetPassword (lazy)	Request link · set new password
/orders	OrderHistory (lazy)	Orders, refunds, cancel, PDF bill, resend e-bill
/account	Account (lazy)	Profile, My coupons, change password, delete account
/support	Support (lazy)	Tickets, replies, open chat
/admin	AdminDashboard (lazy)	Stats and 8 tabs; “Access denied” for non-admins
/about · /contact	About · Contact (lazy)	Brand story · support email, LinkedIn, business details once filled in
/privacy · /terms · /returns	Policy pages (lazy)	Updated 4 Oct (Terms), 10 Oct 2026 (Privacy, Returns)
*	NotFound (lazy)	404 page, noindex

SEO

- `usePageMeta` sets title (“... | ANM-Shop”), description (≤ 160), canonical, OG and Twitter tags.
- Private pages get `noindex`; `robots.txt` blocks them and links the sitemap.
- `index.html` holds defaults for link previews (en-IN, 1200 × 630 og-image).

Performance

- 16 lazy pages; jsPDF, Excel export and Razorpay loaded on demand; chat mounts on first open.
- The footer no longer jumps while a page loads: it appears with the page, and placeholders fill the screen.
- First 4 product cards load eagerly; others lazily.
- Cloudinary sizes: 300 (card), 560 (detail), 96 (cart), 64 (admin) × 2 for sharp screens.
- Preconnect to the API; hashed assets cached for a year.

Consent and analytics

- Banner: “Accept” or “Essential only”; reopened from “Privacy choices”.
- Vercel Analytics and Speed Insights imported only after consent; nothing sent after withdrawal.

Look and feel

- Brand palette (plum `#754656`, gold accents), serif headings.
- Dark mode saved in `anmShopTheme`, applied before first paint.
- Skeleton loaders for lists, grids, stats and tables.

Configuration and operations

12.1 Environment variables

Variable	Where	Purpose
<code>MONGO_URL</code>	backend	MongoDB connection string (required)
<code>JWT_SECRET</code>	backend	Signs JWTs and email-code HMACs; ≥ 32 chars in production
<code>FRONTEND_URL</code>	backend	Allowed origins, comma-separated; the first is the storefront URL in emails and the sitemap
<code>NODE_ENV</code> · <code>PORT</code> · <code>TRUST_PROXY</code>	backend	production checks · port (5000) · proxy hops (1)
<code>RAZORPAY_KEY_ID</code> · <code>RAZORPAY_KEY_SECRET</code>	backend	Payments (checkout returns 503 without them)
<code>RAZORPAY_WEBHOOK_SECRET</code>	backend	Webhook signature
<code>CLOUDINARY_CLOUD_NAME</code> · <code>_API_KEY</code> · <code>_API_SECRET</code>	backend	Image uploads
<code>RESEND_API_KEY</code> · <code>RESEND_FROM_EMAIL</code>	backend	Email through Resend (both needed)
<code>EMAIL_USER</code> · <code>EMAIL_PASS</code>	backend	Gmail SMTP fallback (local development)
<code>GEMINI_API_KEY</code> · <code>GEMINI_MODEL</code> · <code>GEMINI_FALLBACK_MODEL</code>	backend	Chat assistant and optional model overrides
<code>SHIPPING_FEE</code> · <code>FREE_SHIPPING_ABOVE</code>	backend	Rupees; defaults 49 and 499
<code>ADMIN_EMAIL</code>	backend	First admin, made by <code>npm run promote-admin</code> ; later admins are set in the Users tab
<code>VITE_API_URL</code>	frontend	API base, e.g. <code>https://anm-shop-api.onrender.com/api</code> (must be set on Vercel)

The project README repeats the setup and deployment steps.

12.2 Running locally

- 1 From the project root: `npm run install:all` (installs the root tools, backend and frontend).
- 2 Copy `backend/.env.example` to `backend/.env` and fill in at least `MONGO_URL` and `JWT_SECRET` ; add Razorpay test keys, Cloudinary, email and Gemini keys for those features.
- 3 Optional demo data: `cd backend && npm run seed` (44 products; running it again only adds missing ones and never overwrites dashboard changes; `node seed.js -d` removes them).
- 4 `npm run dev` starts the API on port 5000 and Vite on 5173; Vite proxies `/api` to the API.
- 5 `cd backend && npm test` runs the 54 API tests on an in-memory MongoDB with a fake Razorpay; it never reads `.env` (the first run downloads the MongoDB binary). `cd frontend && npm test` runs the 23 component tests in jsdom with the API faked. GitHub Actions runs both on every pull request.
- 6 Sign up, then set `ADMIN_EMAIL` and run `npm run promote-admin` in `backend/` to get an admin account.

12.3 Scripts

Where	Script	Does
root	<code>install:all</code> · <code>dev</code> · <code>dev:server</code> · <code>dev:client</code> · <code>build</code>	install root tools, backend and frontend · run both · API only · web only · build the web app
backend	<code>start</code> · <code>dev</code> · <code>test</code> · <code>seed</code> · <code>promote-admin</code>	node · nodemon · API tests · demo catalogue · first admin
frontend	<code>dev</code> · <code>build</code> · <code>preview</code> · <code>lint</code> · <code>test</code> · <code>csp-hash</code>	Vite dev · CSP check + production build · preview · ESLint · component tests (Vitest) · print the theme-script hash

12.4 Start-up sequence

1. `connectDB()` — exits if MongoDB is unreachable or `MONGO_URL` is missing.
2. `ensureDefaultCategories()` — adds the 7 launch categories when the collection is empty.
3. `migrateEmbeddedReviews()` — moves old 1–5 star reviews into the Review collection (nothing to do once moved).
4. `fixOrderBreakdowns()` — one-time repair of free-shipping coupon orders, recorded as `order-shipping-breakdown-2026-10` in `migrations`.
5. `app.listen(PORT)` . Fix-up errors are logged and never stop the shop from starting.

12.5 Operating checklist

- Razorpay webhook configured to `/api/payment/webhook` for `payment.captured`.
- Check the Orders tab's **Refund problems** filter (admins are also emailed when a cancel or return refund fails) and use Retry refund. Watch Render logs for “AUTOMATIC REFUND FAILED” (checkout refunds: refund in the Razorpay dashboard).
- After editing the inline script in `index.html`, run `npm run csp-hash` and update `vercel.json`; the build stops until they match.
- Keep `FRONTEND_URL` (Render) and `VITE_API_URL` (Vercel) in step if either domain changes; the domains also appear in `index.html`, `robots.txt` and `vercel.json` (rewrite and CSP).
- GitHub Actions runs the tests, lint and build on every pull request, and `main` requires both checks. Before a release, also try a checkout with Razorpay test keys, and have each branch reviewed before it is merged.
- Before taking real orders, fill in the business details and grievance officer in `frontend/src/data/contactInfo.js`, and the same business values in `backend/config/business.js` for the e-bill. A test fails while the two differ or if a value has a character the PDF bill's font can't print (₹, №, Hindi). Update the “Last updated” dates on the Terms and Privacy pages.
- Don't delete files from Cloudinary's `anm-shop/products` folder by hand: it holds every live product photo as well as unused ones.

Project history

Developed between 26 September and 10 October 2026 by Mohil Kumar (GitHub: MOHILKUMAR).

PR	Date	Change
—	26 Sep	Initial commit of the store (73 files)
#1–#3	26 Sep	Payment reliability (webhook, auto-refunds, trust proxy); sign-up takeover, double-charge and e-bill spam fixes; Vercel + Render deploy config
#4–#8	26 Sep	Admin dashboard crash fix; admin search; change password; admin deletes and forgot password; skeleton loading, Users tab and Excel export
#9–#10	26 Sep	Support tickets and AI assistant (Gemini free tier); retries and fallback model
#11	27 Sep	Coupons: admin management, My coupons, checkout pricing; shipping and discount in orders and bills
#12	28 Sep	Separate cart per account on a shared device
#13	1 Oct	Product reviews (Bad / Good / Excellent) with admin hiding
#15	4–9 Oct	Launch checklist: policies, SEO, consent and analytics, speed, spam protection, review fixes
#16	9 Oct	Shared <code>formatInr</code> helper for every rupee amount
#17–#19	9 Oct	Rate-limit follow-ups, run-once order breakdown fix-up, removal of an unused stored shipping fee
#20	9 Oct	This project documentation
#21	10 Oct	Fix known limitations: categories in the database, ranked search, photo galleries, paged admin lists, cancel and return with refunds, account deletion, role management, full CSP, API tests, README, dependency fixes (second edition of this document)
#22–#23	10 Oct	Skip the test MongoDB download on install; the footer no longer jumps while pages load
#24	10 Oct	Code-review fixes: partial-word search, old category links, safer refunds (atomic status change, Razorpay note limit, exact coupon, Refund problems, admin alerts, retry checks Razorpay), account deletion gaps
#25	10 Oct	Admin Tickets tab reloads and follows a ticket after a reply; first frontend tests (Vitest)
#26	10 Oct	Background refund alerts, a 24-hour window and admin checks for account deletion, “paid but being confirmed”, Orders tab loading fix
#27–#28	10 Oct	Known limitations in the README, and small fixes from its review
#29–#30	10 Oct	Third edition of this document, and its sources in <code>docs/source/</code>
#31	10 Oct	CI: GitHub Actions runs the API tests and the shop’s lint, tests and build on every pull request; branch protection then made both checks required on <code>main</code>
#32	10 Oct	E-bill and password-changed emails show India time (the server runs in UTC); tagged <code>v0.9.0</code> , the pre-launch release
#33	10 Oct	Business details and a grievance officer on the Contact page, footer, Terms and Privacy policy, shown once filled in
#34	10 Oct	The seller on the PDF bill and the e-bill email; long PDF bills keep their closing lines on the page
next	10 Oct	This fourth edition of the documentation

Known limitations and next steps

Fixed since the first edition: hard-coded categories, regex-only search, single product photos, unpagged admin lists, no cancel or return, admin-only account deletion, script-only admins, partial CSP, no tests, template README, unused packages. Since the second edition: the footer jumping while pages load, search missing partial words, dead links to renamed categories, refund and account-deletion gaps found in code reviews, and no frontend tests. Since the third edition: no CI, emails showing the server's UTC time, no business details or grievance officer, and long PDF bills losing their last line.

Area	Limitation	Suggested next step
Orders	Refunds are always the full amount; returns are requested through a ticket; no email when an order ships or is delivered; deleting an order record does not refund.	Partial refunds per item, an in-app return request, shipping emails.
Checkout refunds	If an automatic refund for a checkout (item sold out while paying) fails, it must be refunded in the Razorpay dashboard.	Show those payment records with a retry button, as orders have.
Rate limits	In-memory counts reset on restart and are per instance.	Use a shared store (e.g. Redis) before running more than one instance.
Sessions	JWT kept in localStorage for 30 days. httpOnly cookies would be third-party while the API is on another domain.	Serve the API under the shop's domain, then move to httpOnly cookies.
Cart	Browser-only; not synced across devices.	Optional server-side cart for signed-in users.
Admin scale	The Users tab loads every account at once.	Page and search the user list on the server.
SEO	Tags are set in the browser, so link previews show the defaults; domains are hard-coded (also in the CSP).	Pre-render product pages; move domains to config.
AI assistant	Free Gemini tier: quota / busy errors, and content may be used by Google.	Paid tier for production traffic.
Launch	Razorpay is still in test mode, and the business details and grievance officer ship empty (nothing shows until they are filled in, in the site's file and the API's).	Fill in the details, switch Razorpay to live mode, place and refund one real order, then tag <code>v1.0.0</code> .
Testing	54 API tests and 23 frontend component tests (admin Orders, Tickets and Search tabs, Contact and policy pages, PDF bill), run by GitHub Actions on every pull request; checkout, account and other screens are checked by hand.	Cover checkout and account screens.
Small known issues	The README's "Known limitations" section lists the small issues left on purpose (rare, or harmless at the shop's size), each with a workaround: e.g. an unsaved refund not alerting admins, coupon limits reset by re-registering, unused photos left in Cloudinary.	Fix them as the shop grows; the README says when each starts to matter.
Tooling	nodemon (development only) has an advisory whose suggested fix is a 2017 downgrade.	Replace it with <code>node --watch</code> .

Fourth edition, updated on 10 October 2026 from the ANM-Shop repository up to pull request #34.